Studio Nx

S1-09 악기·이펙트 화면 — 함수 설명 합본 (JS 243~263)

2026-10-18

목차

1. ai-notes/javascript/S1-09 - 악기·이펙트 화면/243_initDevices.md
2. ai-notes/javascript/S1-09 - 악기·이펙트 화면/244_readProjectId_loadPlugins.md
3. ai-notes/javascript/S1-09 - 악기·이펙트 화면/245_loadTracks_loadDevices_preloadDevices.md
4. ai-notes/javascript/S1-09 - 악기·이펙트 화면/246_renderTracks.md
5. ai-notes/javascript/S1-09 - 악기·이펙트 화면/247_syncMasterRow_buildTrackRow.md
6. ai-notes/javascript/S1-09 - 악기·이펙트 화면/248_trackInfo_stripInfo_trackById_pluginById.md
7. ai-notes/javascript/S1-09 - 악기·이펙트 화면/249_updateTrack_syncTrackHead.md
8. ai-notes/javascript/S1-09 - 악기·이펙트 화면/250_ensureLiveCells.md
9. ai-notes/javascript/S1-09 - 악기·이펙트 화면/251_renderCells_refreshCells.md
10. ai-notes/javascript/S1-09 - 악기·이펙트 화면/252_instrumentsHtml_effectsHtml.md
11. ai-notes/javascript/S1-09 - 악기·이펙트 화면/253_cardHtml_instrumentMixHtml_effectExtraHtml.md
12. ai-notes/javascript/S1-09 - 악기·이펙트 화면/254_mixerHtml_renderMixerStrips_buildStrip.md
13. ai-notes/javascript/S1-09 - 악기·이펙트 화면/255_clampValue_normOf_fmtValue.md
14. ai-notes/javascript/S1-09 - 악기·이펙트 화면/256_defaultParams_withDefaults_schemaItem.md
15. ai-notes/javascript/S1-09 - 악기·이펙트 화면/257_bindControl_setControlValue.md
16. ai-notes/javascript/S1-09 - 악기·이펙트 화면/258_wireControls_deviceOfCard_paintKnob.md
17. ai-notes/javascript/S1-09 - 악기·이펙트 화면/259_openPluginPicker_renderPickerList.md
18. ai-notes/javascript/S1-09 - 악기·이펙트 화면/260_addDevice_updateDevice_removeDevice_moveEffect.md
19. ai-notes/javascript/S1-09 - 악기·이펙트 화면/261_openDeviceModal_renderModalParams.md
20. ai-notes/javascript/S1-09 - 악기·이펙트 화면/262_loadPresets_applyPreset_toggleBypass.md
21. ai-notes/javascript/S1-09 - 악기·이펙트 화면/263_wireCells_wireDialogs_wireDemoControls.md

JavaScript 243 — 시작 (`initDevices`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 두 모드를 가르고 첫 그림을 그립니다 (FR-241 · FR-251 · FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	242 백엔드 필터 목록 끝 · 244 주소의 프로젝트 번호와 플러그인 표

1. 하는 일

`initDevices` 는 스튜디오 화면이 열릴 때 한 번 불립니다. 주소의 `?projectId=` 를 읽어 **실제 모드**와 **데모 모드**를 가르고, 플러그인 표와 트랙 목록을 읽어 화면을 그린 다음 이벤트를 배선합니다. 실제 모드에서는 칸에 `.is-live` 가 붙어 `sequencer-ui.js` 의 예시 배선과 겹치지 않습니다.

```
initDevices
  ├ readProjectId()          ?projectId= 가 있으면 실제 모드
  ├ wireCells(#lanes)        칸 안 누름과 끌기
  ├ wireDialogs()            고르기 창과 상세 모달
  ├ wireDemoControls()       예시 화면 노브와 페이더만 화면에서 움직이기
  ├ loadPlugins() → loadTracks()
  ├ renderTracks()           #tracks 다시 그리기
  ├ restoreOpenPanels()      담아 둔 펼침 되살리기 (FR-261)
  ├ ensureLiveCells(...)     펼쳐 둔 트랙 칸 채우기
  └ preloadDevices(tracks)   줄 아래 악기 이름 미리 읽기
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js`	페이지가 뜰 때 `initDevices({ onChange: ... })` 를 부릅니다
`onChange`	칸 안에서 값이 바뀌면 arrange 쪽에 알리는 함수입니다

3. 코드

```
export function initDevices(options) {
  const onChange = (options && options.onChange) || function () {};
  projectId = readProjectId();
  active = projectId !== null;

  wireCells(document.getElementById('lanes'), onChange);
  wireDialogs();
  wireDemoControls(document);

  loadPlugins()
    .then(function () {
      if (!active) return null;
```

```
      return loadTracks();
    })
    .then(function () {
      if (!active) return;
      renderTracks();
      // FR-261 — 예시 줄을 갈아 끼운 뒤라, 담아 둔 펼침 상태를 실제 트랙 줄에 다시 적용합니다
      restoreOpenPanels();
      Array.prototype.forEach.call(document.querySelectorAll('#tracks > .sq-track.is-open'), ensureLiveC
ells);
      // 트랙 줄 · 믹서 줄의 악기 이름을 바로 보여 주기 위해 값을 미리 모읍니다
      preloadDevices(tracks).then(syncTrackSubs);
    })
    .catch(function (err) {
      if (err instanceof ApiError && err.status === 401) return;
      active = false;
      showToast('Could not read this project — showing the example screen');
      if (window.console) console.warn('[devices]', err);
    });
}
```

문법 풀이

식	뜻
`options && options.onChange`	넘어온 알림 함수가 없으면 빈 함수를 씁니다
`active = projectId !== null`	주소에 번호가 있으면 실제 모드입니다
`.then(function () { ... })`	앞 단계가 끝난 뒤 이어서 실행합니다
`#tracks > .sq-track.is-open`	펼쳐 둔 트랙 줄만 골라 칸을 채웁니다
`preloadDevices(tracks).then(syncTrackSubs)`	미리 읽기가 끝나면 줄 아래 악기 이름을 다시 씁니다
`err.status === 401` 이면 `return`	로그인이 풀린 경우는 예시 화면으로 바꾸지 않습니다
마지막 `catch`	그 밖의 오류는 예시 화면으로 되돌리고 알립니다

4. 왜 이렇게

선택	이유
플러그인을 먼저 읽음	카드에 플러그인 이름과 파라미터를 붙이려면 표가 먼저 있어야 합니다
그린 뒤 펼침을 복원	`renderTracks` 가 줄을 갈아 끼우므로 담아 둔 펼침은 그 뒤에 적용합니다
실패해도 예시 화면	서버가 없거나 권한이 없어도 화면이 비지 않게 합니다
401 은 그대로 둠	로그인 만료는 로그인 화면이 맡습니다

5. 시험

헤드리스 Chrome 시험에서 계정으로 로그인한 뒤 `?projectId=14431` 로 열어 트랙 5줄과 MASTER 1줄이 그려지고, 줄 아래에 악기 이름이 보이며, 자바스크립트 오류가 0건인 것을 확인했습니다. `projectId` 없이 열면 예시 트랙 2줄과 예시 칸이 그대로 남는 것도 확인했습니다.

JavaScript 244 — 주소의 프로젝트 번호와 플러그인 표 (`readProjectId` · `loadPlugins`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 어느 프로젝트인지 읽고, 붙일 수 있는 플러그인 표를 읽습니다 (FR-241 · FR-244)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	243 시작 · 245 트랙과 장치 읽기

1. 하는 일

두 함수는 화면이 무엇을 그릴지 정하는 첫 재료를 모읍니다. `readProjectId` 는 주소 끝의 `?projectId=12` 에서 `12` 를 꺼내고, 없거나 빈 값이면 `null` 을 돌려줍니다. `loadPlugins` 는 플러그인 표에서 **쓰는 중인 것만** 읽어(FR-241) 모듈 변수 `plugins` 에 담습니다. 카드의 이름, 파라미터 스키마, 고르기 창의 목록이 모두 이 표에서 나옵니다.

2. 언제 쓰나

쓰는 곳	
`initDevices` (243)	모드를 정할 때 `readProjectId`, 첫 재료로 `loadPlugins` 를 부릅니다
`openPluginPicker` (259)	표가 비어 있으면 안내 알림만 띄웁니다

3. 코드

```
export function readProjectId() {
  if (typeof window === 'undefined') return null;
  const raw = new URLSearchParams(window.location.search).get('projectId');
  return raw ? String(raw) : null;
}

export async function loadPlugins() {
  const res = await api.get('/plugins?active_only=true');
  plugins = (res && res.items) || [];
  return plugins;
}
```

문법 풀이

식	뜻
`typeof window === 'undefined'`	화면이 없는 환경(시험 도구)에서도 터지지 않게 합니다
`new URLSearchParams(window.location.search)`	주소의 `?` 뒤 글을 다루기 쉽게 나눕니다
`.get('projectId')`	`projectId` 값 하나를 꺼냅니다. 없으면 `null`

식	뜻
`raw ? String(raw) : null`	빈 글자(`?projectId=`)는 없는 것으로 봅니다
`api.get('/plugins?active_only=true')`	꺼 둔 플러그인을 빼고 읽습니다
`(res && res.items)	

4. 왜 이렇게

선택	이유
주소로 프로젝트를 받음	프로젝트 목록 화면에서 링크 한 줄로 들어올 수 있습니다
`active_only=true`	판매 중지나 시험용 플러그인이 목록에 섞이지 않게 합니다
빈 글자를 `null` 로	실제 모드와 데모 모드의 갈림길이라 빈 값을 확실히 걸러야 합니다

5. 시험

시험 계정으로 로그인한 뒤 `/projects` 에서 "S1-09 UI Test" 의 `project_id` 를 읽어 주소에 붙였고, `?projectId=` 없이 열었을 때는 `null` 이 되어 예시 화면이 그대로 남는 것을 확인했습니다. 플러그인 표는 7종(악기 2 · 이펙트 5)이 읽혔습니다.

JavaScript 245 — 트랙과 장치 읽기, 미리 읽기 (`loadTracks` · `loadDevices` · `preloadDevices` · `syncTrackSubs`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 트랙 목록과 트랙별 악기·이펙트를 서버에서 읽습니다 (FR-245 · FR-251)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	244 주소의 프로젝트 번호와 플러그인 표 · 246 트랙 줄 그리기

1. 하는 일

`loadTracks` 는 `GET /projects/{id}/tracks` 로 트랙 목록을 읽습니다. `loadDevices` 는 트랙 하나의 `GET /tracks/{id}/instruments` 와 `GET /tracks/{id}/plugins` 를 함께 읽어 `devices[track_id]` 에 담습니다. `preloadDevices` 는 **모든 트랙**의 장치를 네 개씩 나눠 미리 읽습니다. 그래야 줄 아래 작은 글과 아래 믹서 줄에 악기 이름이 펼치기 전에도 보입니다. `syncTrackSubs` 는 미리 읽기가 끝난 뒤 그 이름들을 다시 씁니다.

```
loadTracks()                프로젝트 트랙 목록
loadDevices(track)          트랙 하나의 악기 + 이펙트
preloadDevices(tracks)      모든 트랙을 네 개씩 나눠 미리 읽기
      └ syncTrackSubs()     줄 아래 글 · 믹서 줄 이름 다시 쓰기
```

2. 언제 쓰나

쓰는 곳	
`initDevices` (243)	트랙 목록과 미리 읽기에 씁니다
`ensureLiveCells` (250)	값이 아직 없으면 `loadDevices` 로 읽습니다
`addDevice` · `removeDevice` · `moveEffect` (260)	값을 고친 뒤 `refreshCells` 로 다시 그립니다
`toggleBypass` · `toggleSolo` (261)	켜고 끈 뒤 카드만 다시 그립니다

3. 코드

```
export async function loadTracks() {
  const res = await api.get('/projects/' + encodeURIComponent(projectId) + '/tracks');
  tracks = (res && res.items) || [];
  return tracks;
}

export async function loadDevices(track) {
  const id = track.track_id;
  const both = await Promise.all([
    api.get('/tracks/' + id + '/instruments'),
    api.get('/tracks/' + id + '/plugins'),
  ]);
```

```
  devices[id] = { instruments: both[0] || [], effects: both[1] || [] };
  return devices[id];
}

export function preloadDevices(list) {
  const queue = (list || []).slice();
  function worker() {
    const track = queue.shift();
    if (!track) return Promise.resolve();
    if (devices[track.track_id]) return worker();
    return loadDevices(track).catch(function () { return null; }).then(worker);
  }
  const width = Math.min(4, Math.max(1, queue.length));
  const workers = [];
  for (let i = 0; i < width; i += 1) workers.push(worker());
  return Promise.all(workers).then(function () { return undefined; });
}

function syncTrackSubs() {
  tracks.forEach(function (track) {
    const row = rowOf(track);
    if (!row) return;
    const small = row.querySelector('.sq-track__name small');
    if (small) small.textContent = trackInfo(track).sub;
  });
  renderMixerStrips();
}
```

문법 풀이

식	뜻
`encodeURIComponent(projectId)`	주소에 넣을 수 없는 글자를 안전하게 바꿉니다
`Promise.all([...])`	두 요청을 함께 보내고 둘 다 끝날 때까지 기다립니다
`both[0]` · `both[1]`	악기 목록과 이펙트 목록입니다 (각각 배열)
`queue.shift()`	줄 맨 앞 하나를 꺼냅니다
`worker()` 안에서 `worker()`	앞 트랙을 마치면 다음 트랙을 이어서 읽습니다
`Math.min(4, ...)`	한 번에 최대 네 줄기만 돌립니다
`Promise.resolve()`	더 읽을 트랙이 없으면 끝난 것으로 칩니다
`.catch(function () { return null; })`	한 트랙이 실패해도 나머지는 계속 읽습니다

4. 왜 이렇게

선택	이유
트랙별로 따로 읽음	서버에 트랙 장치를 한꺼번에 주는 길이 없습니다
네 줄기로 나눔	트랙이 많아도 요청이 한꺼번에 몰리지 않습니다
이미 읽은 트랙은 건너뜀	펼쳐서 이미 읽은 트랙을 다시 읽지 않습니다
실패를 삼킴	한 트랙의 오류로 줄 전체가 멈추면 안 됩니다
미리 읽기 뒤 이름 다시 쓰기	그리는 시점에는 값이 없어 "No instrument" 로 남기 때문입니다

5. 시험

시험 프로젝트(트랙 6개)에서 줄 아래 글이 처음부터 `Subtractive Synth` 로 보이는 것을 확인했습니다. 미리 읽기를 넣기 전에는 펼치기 전까지 `No instrument` 로 남았습니다.

JavaScript 246 — 트랙 줄 그리기 (`renderTracks`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 서버 트랙으로 #tracks 를 다시 그립니다 (FR-251 · FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	245 트랙과 장치 읽기 · 247 MASTER 줄 맞추기와 트랙 줄 만들기

1. 하는 일

`#tracks` 안의 예시 줄과 예시 칸을 걷어내고 서버 트랙으로 다시 채웁니다. `order_index` 차례대로 늘어놓고, MASTER 는 목록에서 빼서 화면 아래에 이미 있는 예시 MASTER 줄을 값만 맞춰 씁니다(FR-252). 펼침 칸을 복제할 본보기 `#acc-2` 는 지우지 않고 감춰 둡니다. 마지막으로 아래 칸 믹서 줄을 다시 그립니다.

2. 언제 쓰나

쓰는 곳	
`initDevices` (243)	트랙을 읽은 뒤 한 번 부릅니다
트랙을 더하거나 지운 뒤	arrange 쪽이 목록을 바꾸면 다시 불러야 합니다

3. 코드

```
export function renderTracks() {
  const list = document.getElementById('tracks');
  if (!list) return;
  const tpl = document.getElementById('acc-2');

  // 예시 화면의 줄과 칸을 걷어내고, 칸을 복제할 본보기(#acc-2)만 남깁니다
  Array.prototype.forEach.call(list.querySelectorAll(':scope > .sq-track, :scope > .sq-acc'), function
(row) {
    if (row === tpl) return;
    row.remove();
  });
  if (tpl) {
    tpl.hidden = true;
    tpl.classList.remove('is-open');
  }

  const ordered = tracks.slice().sort(function (a, b) { return a.order_index - b.order_index; });
  const master = ordered.filter(function (t) { return t.track_type === 'MASTER'; })[0] || null;
  const rest = ordered.filter(function (t) { return t.track_type !== 'MASTER'; });

  let last = null;
  rest.forEach(function (track) {
    const row = buildTrackRow(track, track.order_index);
    if (last) last.insertAdjacentElement('afterend', row);
```

```
    else if (tpl) list.insertBefore(row, tpl);
    else list.append(row);
    last = row;
  });

  // MASTER 는 #tracks 밖에 있는 예시 줄을 그대로 씁니다 (늘 맨 아래, FR-252)
  const demoMaster = document.querySelector('.sq-track--master');
  if (demoMaster && master) syncMasterRow(master, demoMaster);

  // 예시 리턴 줄(#tracks 밖)만 감춥니다 — 실제 RETURN 트랙 줄은 그대로 둡니다
  Array.prototype.forEach.call(document.querySelectorAll('.sq-track--return'), function (row) {
    if (list.contains(row)) return;
    row.hidden = true;
  });

  renderMixerStrips();
}
```

문법 풀이

식	뜻
`':scope > .sq-track, :scope > .sq-acc'`	#tracks 의 바로 아래 자식만 고릅니다(칸 안쪽은 건드리지 않음)
`if (row === tpl) return;`	복제 본보기 `#acc-2` 는 남깁니다
`.sort(function (a, b) { return a.order_index - b.order_index; })`	순서 번호가 작은 트랙이 위로 옵니다
`.filter(...)[0] || null`	MASTER 는 하나뿐이라 첫 줄만 씁니다
`last.insertAdjacentElement('afterend', row)`	앞에 놓은 줄 바로 뒤에 이어 붙입니다
`list.insertBefore(row, tpl)`	본보기 앞에 끼워 넣습니다
`list.contains(row)`	그 줄이 #tracks 안에 있는지 봅니다

4. 왜 이렇게

선택	이유
본보기 칸을 남김	실제 트랙마다 칸을 복제해 쓰므로 원본이 필요합니다
MASTER 는 예시 줄 재사용	화면 맨 아래 고정 자리와 모양이 이미 잡혀 있습니다 (FR-252)
바깥 예시 리턴 줄만 감춤	실제 RETURN 트랙은 목록 안에 세워야 하기 때문입니다
정렬을 새 배열로	원본 `tracks` 차례는 그대로 두어 다른 곳에서도 씁니다

5. 시험

시험 프로젝트로 열었을 때 `#tracks > .sq-track` 이 5줄(리턴 포함), MASTER 예시 줄이 보이고, 예시 `#acc-1` 이 사라지며 본보기 `#acc-2` 가 감춰진 상태로 남는 것을 확인했습니다.

JavaScript 247 — MASTER 줄 맞추기와 트랙 줄 만들기 (`syncMasterRow` · `buildTrackRow`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 트랙 한 줄의 겉모양(색 · 배지 · M · S · ⏻)을 만듭니다 (FR-251 · FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	246 트랙 줄 그리기 · 248 줄 아래 글 만들기

1. 하는 일

`buildTrackRow` 는 트랙 한 줄(`li.sq-track`)을 studioNx.html 의 예시 줄과 같은 모양으로 만듭니다. 이름, 종류 배지(AUDIO · MIDI · RET · GROUP), 색, 펼침 V 버튼, M · S · ● 버튼, 겹 수가 담깁니다. `syncMasterRow` 는 화면에 이미 있는 예시 MASTER 줄에 실제 MASTER 트랙의 번호와 상태를 옮겨 붙입니다.

2. 언제 쓰나

쓰는 곳	
`renderTracks` (246)	목록의 트랙마다 `buildTrackRow`, MASTER 에 `syncMasterRow` 를 씁니다
`sequencer.js` · arrange	트랙을 새로 만들 때 같은 모양이 필요합니다

3. 코드

```
export function buildTrackRow(track, i, master) {
  const li = document.createElement('li');
  li.className = 'sq-track';
  if (master || track.track_type === 'MASTER') li.classList.add('sq-track--master');
  if (track.track_type === 'RETURN') li.classList.add('sq-track--return');
  if (track.track_type === 'GROUP') li.classList.add('sq-track--group');
  li.dataset.trackId = String(track.track_id);
  li.dataset.color = track.track_type === 'AUDIO' ? 'audio' : String((i % 5) + 1);
  if (track.is_enabled === false) li.classList.add('is-off');
  if (track.is_muted) li.classList.add('is-muted');
  const num = master ? '' : (track.order_index + 1) + '. ';
  let badge = '';
  if (master) badge = '<span class="sq-badge">MAIN</span>';
  else if (track.track_type === 'RETURN') badge = '<span class="sq-badge is-group">RET</span>';
  else if (track.track_type === 'GROUP') badge = '<span class="sq-badge is-group">GROUP</span>';
  // ...(MIDI · AUDIO 배지)...
  const info = trackInfo(track);
  li.innerHTML = '...'
    + '<span class="sq-track__name"><b>' + esc(num + track.name) + ' ' + badge + '</b><small>' + esc(inf
o.sub) + '</small></span>'
    + '<span class="sq-msr"><button type="button" class="is-mute" aria-pressed="' + String(!!track.is_mu
ted) + '" aria-label="Mute">M</button>'
```

```
    + (master ? '' : '<button type="button" class="is-solo" aria-pressed="' + String(!!track.is_solo) +
'" aria-label="Solo">S</button>')
    + (master || track.track_type === 'RETURN' ? '' : '<button type="button" class="is-arm" aria-pressed
="false" aria-label="Record arm">●</button>')
    + '</span>\n            </div>\n            <div class="sq-track__lane"><span class="sq-track__not
e">' + esc(info.note) + '</span></div>';
  return li;
}

function syncMasterRow(track, row) {
  row.dataset.trackId = String(track.track_id);
  const small = row.querySelector('.sq-track__name small');
  if (small) small.textContent = 'Main output';
  const power = row.querySelector('.sq-power');
  if (power) power.setAttribute('aria-pressed', String(track.is_enabled !== false));
  const mute = row.querySelector('.is-mute');
  if (mute) mute.setAttribute('aria-pressed', String(!!track.is_muted));
  row.classList.toggle('is-muted', !!track.is_muted);
}
```

문법 풀이

식	뜻
`li.dataset.color`	CSS 가 색을 고르는 데 쓰는 `data-color` 가 됩니다
`i % 5`	트랙 차례를 1~5 색으로 돌립니다
`master ? '' : ...`	MASTER 에는 솔로와 녹음 버튼을 만들지 않습니다
`aria-pressed`	버튼이 눌린 상태인지 화면 낭독기에 알립니다
`esc(...)`	이름에 `<` 같은 글자가 있어도 그대로 글자로 보이게 합니다
`Row.classList.toggle('is-muted', 조건)`	조건이 참이면 붙이고 거짓이면 뗍니다

4. 왜 이렇게

선택	이유
예시 줄과 같은 클래스	CSS 와 arrange 배선을 그대로 다시 씁니다
종류별 배지	한눈에 AUDIO · MIDI · 리턴 · 그룹을 구분합니다
MASTER 에 번호 없음	MASTER 는 차례 번호를 매기지 않습니다
`aria-pressed` 로 상태	저장 값과 화면을 늘 맞춰 두면 눌림 표시가 어긋나지 않습니다

5. 시험

색 규칙 시험에서 AUDIO 트랙 1개와 나머지 1~5 색 4개가 나오는 것을 확인했고, MASTER 줄 이름이 MASTER 이고 숨겨지지 않은 것을 확인했습니다.

JavaScript 248 — 줄 아래 글 만들기 (`trackInfo` · `stripInfo` · `trackById` · `pluginById`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 트랙 줄과 믹서 줄에 쓸 이름 글을 만듭니다 (FR-251 · FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	247 MASTER 줄 맞추기와 트랙 줄 만들기 · 249 트랙 값 저장

1. 하는 일

트랙 줄 아래에는 그 트랙이 무엇을 연주하는지 짧은 글이 붙습니다. `trackInfo` 는 그 글(`sub`)과 이펙트 개수(`fx`)를 만들고, `stripInfo` 는 아래 칸 믹서 줄에 쓸 제목(`2. Bass Synth`)과 같은 글을 만듭니다. 둘 다 트랙 종류에 따라 다르게 씁니다. 악기가 여럿이면 뒤에 `+1` 처럼 남은 수를 덧붙입니다. `trackById` 와 `pluginById` 는 번호로 트랙과 플러그인을 찾는 작은 도우미입니다.

트랙 종류	줄 아래 글 (`trackInfo.sub`)	믹서 줄 글 (`stripInfo.sub`)
MIDI (악기 있음)	🎹 `Subtractive Synth`	`Subtractive Synth`
MIDI (악기 없음)	🎹 `No instrument`	`No instrument`
AUDIO	🎵 `Audio track`	`Audio`
RETURN	`Return — receives sends`	`Return`
GROUP	`Group track`	`No instrument`
MASTER	`Main output`	`Main output`

2. 언제 쓰나

쓰는 곳	
`buildTrackRow` (247)	줄을 만들 때 `trackInfo` 를 씁니다
`buildStrip` (254)	아래 칸 믹서 줄에 `stripInfo` 를 씁니다
`syncTrackSubs` (245) · `refreshCells` (251)	값이 바뀐 뒤 이 글만 다시 씁니다
`wireCells` (262) · `deviceOfCard` (258)	눌린 카드가 어느 트랙 것인지 `trackById` 로 찾습니다

3. 코드

```
/** 트랙 줄 아래 작은 글 — 악기 이름 (+ 겹 수) 또는 트랙 종류 */
function trackInfo(track) {
  const d = devices[track.track_id] || { instruments: [], effects: [] };
  const first = d.instruments[0];
  const p = first ? pluginById(first.plugin_id) : null;
  let sub;
  if (track.track_type === 'MASTER') sub = 'Main output';
```

```
  else if (track.track_type === 'RETURN') sub = 'Return — receives sends';
  else if (track.track_type === 'GROUP') sub = 'Group track';
  else if (track.track_type === 'AUDIO') sub = '🎵 Audio track';
  else sub = p ? '🎹 ' + p.name : '🎹 No instrument';
  if (d.instruments.length > 1) sub += ' +' + (d.instruments.length - 1);
  return { sub: sub, fx: d.effects.length };
}

/** 트랙 줄 · 아래 칸 믹서 줄에서 쓸 이름 정보 */
function stripInfo(track) {
  const d = devices[track.track_id] || { instruments: [], effects: [] };
  const first = d.instruments[0];
  const p = first ? pluginById(first.plugin_id) : null;
  const title = track.track_type === 'MASTER' ? 'MASTER' : (track.order_index + 1) + '. ' + track.name;
  let sub;
  if (track.track_type === 'MASTER') sub = 'Main output';
  else if (p) sub = p.name;
  else if (track.track_type === 'AUDIO') sub = 'Audio';
  else if (track.track_type === 'RETURN') sub = 'Return';
  else sub = 'No instrument';
  if (d.instruments.length > 1) sub += ' +' + (d.instruments.length - 1);
  return { title: title, sub: sub, fx: d.effects.length };
}

/** track_id 로 트랙 찾기 */
function trackById(id) {
  for (let i = 0; i < tracks.length; i += 1) {
    if (String(tracks[i].track_id) === String(id)) return tracks[i];
  }
  return null;
}

/** id 로 플러그인 찾기 */
function pluginById(id) {
  for (let i = 0; i < plugins.length; i += 1) {
    if (plugins[i].plugin_id === Number(id)) return plugins[i];
  }
  return null;
}
```

문법 풀이

식	뜻
`devices[track.track_id] || { instruments: [], effects: [] }`	아직 안 읽은 트랙도 빈 값으로 안전하게 다룹니다
`if / else if` 사슬	트랙 종류가 여럿이라 하나만 골라 씁니다
`p ? '🎹 ' + p.name : '🎹 No instrument'`	악기 표에 있으면 이름, 없으면 안내 글입니다
`sub += ' +' + (d.instruments.length - 1)`	겹 악기가 남아 있으면 `+1` 처럼 붙입니다
`String(...) === String(...)`	서버가 숫자와 글자를 섞어 주므로 글자로 맞춰 봅니다
`plugin_id === Number(id)`	플러그인 번호는 숫자라 숫자로 바꿔 봅니다

4. 왜 이렇게

선택	이유
두 함수를 나눔	줄 아래 글과 믹서 줄 글의 문구가 조금씩 다릅니다
종류별 문구	악기 없는 AUDIO 트랙에 "No instrument" 를 쓰면 오해를 줍니다
`devices` 를 먼저 봄	악기 이름을 서버에 또 묻지 않고 담아 둔 값으로 씁니다
도우미를 따로 둠	같은 찾기를 여러 곳에서 써서 한 곳만 고치면 되게 합니다

5. 시험

시험 프로젝트에서 Bass Synth 줄 아래가 `Subtractive Synth`(겹 1개라 `+` 없음), Vocal 줄이 `Audio track`, Drums 줄이 `Group track`, Reverb Bus 줄이 `Return – receives sends` 로 보이는 것을 확인했습니다. 아래 칸 믹서 줄 제목은 `1. Master` 를 뺀 `2. Bass Synth` 형식이었습니다.

JavaScript 249 — 트랙 값 저장 (`updateTrack` · `syncTrackHead`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 트랙 머리의 M · S · ⏻ 를 서버에 저장합니다 (FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	248 줄 아래 글 만들기 · 250 펼친 트랙의 칸 채우기

1. 하는 일

`updateTrack` 은 트랙 값 하나를 `PATCH /tracks/{id}` 로 보내고, 돌아온 값을 담아 둔 트랙 객체에 덮어씁니다. `syncTrackHead` 는 트랙 줄 머리의 ⏻ · M · S 버튼이 지금 눌린 상태를 읽어 그대로 저장합니다. 화면 모양은 arrange 쪽이 이미 바꾸므로, 이 함수는 **화면을 읽어 서버에 맞추는** 일만 합니다. `rowOf` 와 `pressed` 는 그때 쓰는 작은 도우미입니다.

```
M 클릭 → arrange 가 화면을 먼저 바꿈
        └ syncTrackHead(track)   aria-pressed 를 읽어
              └ updateTrack(track, {is_muted:true})   PATCH /tracks/4639
```

2. 언제 쓰나

쓰는 곳	
`sequencer-arrange.js`	트랙 머리 버튼을 누른 뒤 `syncTrackHead` 를 부릅니다
`saveDeviceModal` (261) · `toggleBypass` (261)	장치 값을 저장할 때 `updateDevice` 를 씁니다(249 의 트랙 판과 짝)
`syncTrackSubs` · `refreshCells`	줄을 찾을 때 `rowOf` 를 씁니다

3. 코드

```
/** 값 고치기 — PATCH /tracks/{id} (볼륨 · 팬 · M · S · 켜기) */
export async function updateTrack(track, patch) {
  const saved = await api.patch('/tracks/' + track.track_id, patch);
  Object.assign(track, saved);
  return saved;
}

/** 트랙 머리의 M · S · ⏻ 를 눌렀을 때 서버에 저장 (화면은 arrange 가 이미 바꿔 둠 → 다음 틱에 읽음) */
export function syncTrackHead(track) {
  const row = rowOf(track);
  if (!row || !active) return;
  const head = row.querySelector('.sq-track__head');
  if (!head) return;
  const patch = {
    is_enabled: pressed(head, '.sq-power'),
    is_muted: pressed(head, '.is-mute'),
  };
```

```
  if (head.querySelector('.is-solo')) patch.is_solo = pressed(head, '.is-solo');
  updateTrack(track, patch).catch(warn);
}

/** 트랙 줄 찾기 */
function rowOf(track) {
  return document.querySelector('.sq-track[data-track-id="' + track.track_id + '"]');
}

/** 버튼이 눌려 있나 */
function pressed(scope, sel) {
  const b = scope.querySelector(sel);
  return !!b && b.getAttribute('aria-pressed') === 'true';
}
```

문법 풀이

식	뜻
`api.patch('/tracks/' + track.track_id, patch)`	보낸 값만 바꾸는 요청입니다
`Object.assign(track, saved)`	서버가 돌려준 값으로 담아 둔 트랙을 갱신합니다
`if (!row || !active) return;`	데모 모드이거나 줄이 없으면 저장하지 않습니다
`head.querySelector('.is-solo')`	MASTER 나 RETURN 에는 솔로 버튼이 없어 있을 때만 넣습니다
`pressed(head, '.sq-power')`	`aria-pressed` 가 `'true'` 인지 봅니다
`.catch(warn)`	저장이 실패해도 경고만 남기고 화면은 그대로 둡니다

4. 왜 이렇게

선택	이유
화면을 먼저 바꾸고 저장	눌림 표시가 즉시 반응해야 손맛이 좋습니다. 서버 왕복을 기다리지 않습니다
저장 실패를 삼킴	잠깐 서버가 없어도 화면 조작은 이어져야 합니다
돌아온 값을 덮어씀	서버가 기본값을 채워 줄 수 있어 그 값을 기준으로 삼습니다
`updateDevice` 와 짝	트랙 값과 장치 값을 같은 방식으로 저장해 코드를 읽기 쉽게 합니다

5. 시험

시험 프로젝트의 Bass Synth M 을 눌러 `aria-pressed` 가 `true` 가 되고 `GET /tracks/4639` 의 `is_muted` 가 `true` 로 저장되는 것을 확인했습니다. 같은 트랙에서 다시 눌러 `false` 로 되돌렸습니다.

JavaScript 250 — 펼친 트랙의 칸 채우기 (`ensureLiveCells` · `accOf` · `cellOf` · `setCellBody`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 펼친 트랙의 네 칸을 실제 값으로 채웁니다 (FR-241)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	249 트랙 값 저장 · 251 펼침 칸 안 네 칸 그리기

1. 하는 일

트랙을 펼치면 예시 칸이 나오는데, 실제 모드에서는 그 칸에 `.is-live` 를 붙이고 **서버 값으로 다시 채워야** 합니다. `ensureLiveCells` 가 그 일을 맡습니다. 값이 아직 없으면 먼저 읽고, 읽은 뒤 칸을 다시 그립니다. `accOf` 는 트랙 줄에 붙은 펼침 칸을 찾고, `cellOf` 는 칸 안에서 이름표로 네 칸 중 하나를 찾고, `setCellBody` 는 칸 머리 글(h3)은 그대로 두고 안쪽만 새로 그립니다.

2. 언제 쓰나

쓰는 곳	
`initDevices` (243)	이미 펼쳐 둔 트랙(`is-open`)마다 부릅니다
`sequencer-arrange.js`	V 버튼으로 트랙을 펼친 뒤 부릅니다
`refreshCells` (251) · `saveDeviceModal` (261)	칸을 찾을 때 `accOf` 를 씁니다

3. 코드

```
export function ensureLiveCells(track) {
  const acc = accOf(track);
  if (!acc) return null;
  acc.classList.add('is-live');
  if (!devices[track.track_id]) {
    renderCells(acc, track);
    // 값이 없으면 먼저 읽고, 읽은 뒤 이 칸만 다시 그립니다 (줄 아래 글 · 믹서 줄까지)
    loadDevices(track).then(function () {
      refreshCells(track);
    }).catch(warn);
    return acc;
  }
  renderCells(acc, track);
  return acc;
}

/** 트랙의 펼침 칸(.sq-acc) 찾기 */
function accOf(track) {
  const row = rowOf(track);
```

```
  if (!row) return null;
  const btn = row.querySelector('.sq-expand');
  const id = btn && btn.getAttribute('aria-controls');
  if (id) {
    const acc = document.getElementById(id);
    if (acc) return acc;
  }
  const sib = row.nextElementSibling;
  return sib && sib.classList.contains('sq-acc') ? sib : null;
}

/** 칸 머리(h3)는 그대로 두고 안쪽만 새 HTML 로 바꾸기 */
function setCellBody(cell, html) {
  if (!cell) return;
  const h3 = cell.querySelector(':scope > h3');
  Array.prototype.forEach.call(cell.children, function (child) {
    if (child !== h3) child.remove();
  });
  const box = document.createElement('div');
  box.className = 'sq-cell__body';
  box.innerHTML = html;
  cell.append(box);
}

/** 칸 찾기 — aria-label 이름 여러 개 중 먼저 있는 것 */
function cellOf(acc, names) {
  for (let i = 0; i < names.length; i += 1) {
    const cell = acc.querySelector('[aria-label="' + names[i] + '"]');
    if (cell) return cell;
  }
  return null;
}
```

문법 풀이

식	뜻
`acc.classList.add('is-live')`	실제 모드 표시입니다. 예시 배선과 겹치지 않게 합니다
`if (!devices[track.track_id])`	아직 안 읽은 트랙인지 봅니다
`loadDevices(track).then(...)`	읽은 뒤 `refreshCells` 로 다시 그립니다
`btn.getAttribute('aria-controls')`	V 버튼이 가리키는 칸 번호를 꺼냅니다
`row.nextElementSibling`	바로 다음 형제가 펼침 칸입니다
`':scope > h3'`	칸의 바로 아래 h3 만 고릅니다
`if (child !== h3) child.remove()`	머리 글은 남기고 나머지를 걷어냅니다
`names` 배열	이름표가 'Instruments' 와 'Instrument' 두 가지라 차례로 봅니다

4. 왜 이렇게

선택	이유
먼저 그리고 나중에 다시 그림	빈 칸이라도 먼저 띄워야 눌러도 반응이 없는 듯한 느낌을 주지 않습니다

선택	이유
머리 글을 남김	제목과 배지(예: `MIDI`, `same notes → layers`)는 값과 상관없습니다
`aria-controls` 를 먼저 봄	HTML 에 적힌 짝이 가장 믿을 만합니다. 없으면 이웃으로 찾습니다
`is-live` 표시	한 화면에 예시 칸과 실제 칸이 섞이므로 이 표시로 갈라 씁니다

5. 시험

`?projectId=14431` 로 열어 Bass Synth 를 펼쳤을 때 칸에 `.is-live` 가 붙고, 악기 카드 1장과 이펙트 카드 2장(Compressor · EQ3)이 그려지며, 그 전에 `Loading…` 이 잠깐 보이는 것을 확인했습니다. 값이 이미 읽힌 뒤 두 번째로 펼치면 곧바로 카드가 그려졌습니다.

JavaScript 251 — 펼침 칸 안 네 칸 그리기 (`renderCells` · `refreshCells`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: Instrument · Effects · Connect · Mixer 네 칸을 값으로 채웁니다 (FR-241 · FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	250 펼친 트랙의 칸 채우기 · 252 악기 칸과 이펙트 칸 HTML

1. 하는 일

`renderCells` 는 칸 하나(`.sq-acc`) 안의 네 칸을 모두 다시 그립니다. 악기 칸은 트랙 종류에 따라 MIDI 면 Nyxi 카드 목록, AUDIO 나 그룹이면 안내 글을 넣습니다. 이펙트 칸은 카드 사슬을, 믹서 칸은 페이더와 팬을 넣습니다. Connect 칸은 다음 작업(S1-10) 예고 글만 넣습니다. 마지막에 칸 안 조작을 연결하고(`wireControls`) 미터를 돌립니다. `refreshCells` 는 값이 바뀐 뒤 그 트랙의 칸만, 그리고 줄 아래 글과 믹서 줄을 다시 그립니다.

2. 언제 쓰나

쓰는 곳	
`ensureLiveCells` (250)	칸을 처음 채울 때 부릅니다
`preloadDevices` 뒤	`refreshCells` 로 줄 아래 글까지 갱신합니다(`syncTrackSubs`)
`addDevice` · `removeDevice` · `moveEffect` (260)	붙이거나 뗀 뒤 다시 그립니다
`saveDeviceModal` (261) · `applyPreset` (261)	저장한 값으로 다시 그립니다

3. 코드

```
export function renderCells(acc, track) {
  const loading = !devices[track.track_id];
  const kind = track.track_type;
  const midi = kind === 'MIDI';
  const inst = cellOf(acc, ['Instruments', 'Instrument']);
  if (inst) {
    const h3 = inst.querySelector(':scope > h3');
    if (h3) {
      h3.innerHTML = midi
        ? 'INSTRUMENTS <span class="sq-badge">MIDI</span><span class="sq-cell__note">same notes → layers
</span>'
        : 'INSTRUMENTS <span class="sq-badge">' + esc(kind) + '</span>';
    }
    if (loading) setCellBody(inst, '<p class="sq-cell__hint">Loading…</p>');
    else if (midi) setCellBody(inst, instrumentsHtml(track));
    else setCellBody(inst, '<p class="sq-cell__hint">' + (kind === 'AUDIO'
      ? 'Drop a clip on this track to load a sample — sample settings come with S1-10'
      : 'This track has no instrument — effects and mixer still work') + '</p>');
```

```
  }
  const fx = cellOf(acc, ['Effects']);
  if (fx) {
    const h3 = fx.querySelector(':scope > h3');
    if (h3) h3.innerHTML = 'EFFECTS <span>signal → · drag ⠿ to reorder</span>';
    setCellBody(fx, loading ? '<p class="sq-cell__hint">Loading…</p>' : effectsHtml(track));
  }
  const mix = cellOf(acc, ['Mixer']);
  if (mix) setCellBody(mix, mixerHtml(track));
  const conn = cellOf(acc, ['Connect to another']);
  if (conn) {
    const h3 = conn.querySelector(':scope > h3');
    if (h3) h3.textContent = 'CONNECT TO ANOTHER';
    setCellBody(conn, '<p class="sq-cell__hint">Linking this track to another one is saved in the next w
ork unit (S1-10, FR-247)</p>');
  }
  wireControls(acc, track);
  startMeters();
}

/** 고친 뒤 그 트랙의 칸만 다시 그리기 (구조가 바뀔 때 — 붙이기 · 떼기 · 순서 · 프리셋) */
function refreshCells(track) {
  const acc = accOf(track);
  if (acc && acc.classList.contains('is-live')) renderCells(acc, track);
  const row = rowOf(track);
  if (row) {
    const small = row.querySelector('.sq-track__name small');
    if (small) small.textContent = trackInfo(track).sub;
  }
  renderMixerStrips();
}
```

문법 풀이

식	뜻
`const loading = !devices[track.track_id]`	값이 없으면 `Loading…` 을 씁니다
`midi ? ... : ...`	MIDI 트랙만 악기 카드를 그립니다
`esc(kind)`	트랙 종류 글자를 안전하게 배지에 넣습니다
`cellOf(acc, ['Instruments', 'Instrument'])`	이름이 두 가지라 두 후보를 넘깁니다
`setCellBody(fx, loading ? ... : effectsHtml(track))`	읽는 중이면 안내 글, 끝났으면 카드 사슬
`acc.classList.contains('is-live')`	실제 칸일 때만 다시 그립니다
`renderMixerStrips()`	아래 칸 믹서 줄도 함께 갱신합니다

4. 왜 이렇게

선택	이유
트랙 종류로 가름	AUDIO 트랙에 악기 카드 자리를 만들면 빈 칸처럼 보입니다
Connect 칸을 예고로	링크 저장은 다음 작업이라 빈 칸 대신 설명을 둡니다 (FR-247)
`refreshCells` 를 나눔	구조가 바뀔 때만 전체를 다시 그리고, 노브 값만 바뀔 때는 그리지 않습니다

선택	이유
줄 아래 글도 함께	악기를 붙이거나 떼면 이름 글이 바뀌어야 합니다

5. 시험

시험 프로젝트에서 MIDI 트랙(Bass Synth)을 펼치면 악기 칸에 `INSTRUMENTS MIDI same notes → layers` 머리와 카드 1장, 이펙트 칸에 `EFFECTS signal → · drag ⠿ to reorder` 머리와 카드 2장이 나오는 것을 확인했습니다. 이펙트를 뗀 뒤 카드가 1장으로 줄고 줄 아래 글과 믹서 줄 FX 수가 함께 바뀌는 것도 확인했습니다.

JavaScript 252 — 악기 칸과 이펙트 칸 HTML (`instrumentsHtml` · `effectsHtml` · `sorted` · `artHtml` · `primaryItems`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 칸 안에 들어갈 카드 묶음과 + 버튼 글을 만듭니다 (FR-245 · FR-263)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	251 펼침 칸 안 네 칸 그리기 · 253 Nyxi 카드 한 장

1. 하는 일

`instrumentsHtml` 은 악기 카드를 `slot_index` 차례로 늘어놓고 끝에 `+ Add instrument` 를 붙입니다. MIDI 트랙에서 같은 노트를 여러 악기가 함께 연주할 수 있게 하려는 것입니다(FR-245). `effectsHtml` 은 카드 사이에 `→` 를 넣어 소리 흐름을 보여 주고 끝에 `+` 를 붙입니다(FR-263). `sorted` 는 두 목록을 칸 번호 차례로 정리하고, `artHtml` 은 카드 아트를, `primaryItems` 는 카드에 올릴 핵심 노브를 고릅니다.

```
악기 칸 : [카드][카드] + Add instrument
이펙트 칸: [카드] → [카드] → +
```

2. 언제 쓰나

쓰는 곳	
`renderCells` (251)	악기 칸과 이펙트 칸의 속을 채울 때 씁니다
`cardHtml` (253)	`artHtml` 과 `primaryItems` 를 씁니다

3. 코드

```
/** 악기 칸 — Nyxi 카드 여러 장 + + (한 트랙에서 같은 노트를 여러 악기가 함께 연주, FR-245) */
function instrumentsHtml(track) {
  const list = sorted((devices[track.track_id] || { instruments: [] }).instruments);
  const cards = list.map(function (d) { return cardHtml('INSTRUMENT', track, d); }).join('');
  return '<div class="sq-istack sq-istack--cards">' + cards
    + '<div class="sq-dev-empty">' + (list.length ? '' : 'No instrument — the note clips play nothing.')
    + '<button type="button" class="is-dev-add" data-kind="INSTRUMENT">+ Add instrument</button></div></
div>';
}

/** 이펙트 칸 — 카드 → 카드, 마지막에 + (FR-263) */
function effectsHtml(track) {
  const list = sorted((devices[track.track_id] || { effects: [] }).effects);
  const parts = [];
  list.forEach(function (d, i) {
    if (i) parts.push('<li class="sq-flow" aria-hidden="true">→</li>');
    parts.push(cardHtml('EFFECT', track, d));
```

```
  });
  parts.push('<li><button type="button" class="sq-fx-add is-dev-add" data-kind="EFFECT" aria-label="Add
effect after the last one">+</button></li>');
  return '<ol class="sq-chain">' + parts.join('') + '</ol>'
    + '<p class="sq-cell__hint">Parallel branches and the PATCHER board come later (FR-265)</p>';
}

/** slot_index 순서로 */
function sorted(list) {
  return list.slice().sort(function (a, b) { return a.slot_index - b.slot_index; });
}

/** 카드 아트 — card_asset 이 있으면 그림, 없으면 이름 두 글자 (시드 7종은 모두 NULL → 글자) */
function artHtml(plugin) {
  if (plugin && plugin.card_asset) return '<img src="' + esc(plugin.card_asset) + '" alt="">';
  const name = plugin ? plugin.name : 'PL';
  return esc(name.replace(/[^A-Za-z0-9]/g, '').slice(0, 2).toUpperCase() || 'PL');
}

/** 카드에 올릴 핵심 파라미터 — primary=true 노브를 먼저, 2~4개 (없으면 앞의 노브 2개) */
function primaryItems(plugin) {
  const schema = (plugin && plugin.param_schema) || [];
  const knobs = schema.filter(function (it) { return it.type === 'knob'; });
  const prim = knobs.filter(function (it) { return it.primary === true; });
  const pick = prim.length ? prim : knobs;
  return pick.slice(0, 4);
}
```

문법 풀이

식	뜻
`sorted(...)`	칸 번호(slot_index) 차례로 정리합니다
`list.map(function (d) { return cardHtml(...); }).join('')`	카드 글을 하나로 이어 붙입니다
`(list.length ? '' : 'No instrument ...')`	악기가 없을 때만 안내 글을 넣습니다
`if (i) parts.push(...)`	첫 카드 앞에는 화살표를 넣지 않습니다
`data-kind="INSTRUMENT"`	`+` 버튼이 무엇을 붙일지 적어 둡니다
`list.slice().sort(...)`	원본 목록 차례는 그대로 두고 사본만 정리합니다
`name.replace(/[^A-Za-z0-9]/g, '')`	글자가 아닌 것을 지워 아트 글자를 만듭니다
`pick.slice(0, 4)`	핵심 노브는 최대 네 개입니다

4. 왜 이렇게

선택	이유
악기는 나란한 카드 묶음	여러 악기가 같은 노트를 함께 연주하므로 차례가 곧 겹 차례입니다
이펙트는 사슬 모양	소리가 카드를 차례로 지나므로 `→` 로 흐름을 보여 줍니다
끝에 `+` 를 둠	새 카드가 늘 마지막 칸에 붙기 때문입니다 (FR-263)
아트가 없으면 두 글자	시드 플러그인 7종은 그림 파일이 없어 빈 자리를 글자로 채웁니다
`primary` 를 먼저 봄	카드는 핵심 노브만 보여 주고, 나머지는 상세 모달에서 다룹니다

5. 시험

시험 프로젝트에서 Bass Synth 칸에 악기 카드 1장과 `+ Add instrument`, 이펙트 카드 2장이 `Compressor → EQ3` 차례로 화살표와 함께 나오는 것을 확인했습니다. EQ3 카드의 핵심 노브는 Low · Mid · High 3개였고, 플러그인 표의 `primary` 표시와 일치했습니다.

JavaScript 253 — Nyxi 카드 한 장 (`cardHtml` · `instrumentMixHtml` · `effectExtraHtml`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: Nyxi 카드 한 장을 만듭니다 (FR-2181)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	252 악기 칸과 이펙트 칸 HTML · 254 믹서 칸과 아래 믹서 줄

1. 하는 일

카드 한 장에는 아트 자리, 이름, 바이패스 버튼(이펙트만), 떼기 버튼(X), 핵심 파라미터 노브 2~4개, 프리셋 이름이 담깁니다. 악기 카드에는 아래 줄로 겹 볼륨 · 팬 · 솔로가 더 붙고(`instrumentMixHtml`), 이펙트 카드에는 미니 미터가 붙습니다(`effectExtraHtml`). 카드에는 저장 값을 알려 주는 표시 두 개가 붙습니다. `data-device-kind` 는 그 카드가 악기인지 이펙트인지, `data-track-plugin-id` 또는 `data-track-instrument-id` 는 서버 줄 번호입니다. 이 번호는 나중에 값을 저장하거나 떼어낼 때 쓰입니다.

2. 언제 쓰나

쓰는 곳	
`instrumentsHtml` (252) · `effectsHtml` (252)	카드마다 `cardHtml` 을 부릅니다
`wireControls` (258)	`data-device-kind` 와 줄 번호로 장치를 찾습니다
`wireCells` (262)	카드 안에서 눌린 버튼을 이 표시로 알아봅니다

3. 코드

```
export function cardHtml(kind, track, d) {
  const p = pluginById(d.plugin_id);
  const name = p ? p.name : 'Plugin #' + d.plugin_id;
  const values = withDefaults(p, d.params);
  const prim = primaryItems(p);
  const bypassed = kind === 'EFFECT' && d.is_bypassed;
  const idAttr = kind === 'EFFECT'
    ? ' data-track-plugin-id="' + d.track_plugin_id + '"'
    : ' data-track-instrument-id="' + d.track_instrument_id + '"';
  const knobs = prim.map(function (item) {
    return '<span class="sq-knob" role="slider" tabindex="0" data-param-key="' + esc(item.key) + '"'
      + ' aria-label="' + esc(item.name) + '" aria-valuemin="' + numOr(item.min, 0) + '"'
      + ' aria-valuemax="' + numOr(item.max, 1) + '" aria-valuenow="' + numOr(values[item.key], 0) +
'">'
      + '<i style="--a:' + knobAngle(item, values[item.key]) + 'deg"></i>'
      + '<span class="sq-knob__val">' + esc(fmtValue(item, values[item.key])) + '</span>'
      + '<span class="sq-knob__name">' + esc(item.name) + '</span></span>';
  }).join('');
```

```
  const mix = kind === 'INSTRUMENT' ? instrumentMixHtml(d) : effectExtraHtml();
  return '<li class="sq-fx sq-fx--dev' + (bypassed ? ' is-bypassed' : '') + '" draggable="' + String(kin
d === 'EFFECT') + '"'
    + ' data-device-kind="' + kind + '"' + idAttr + '>'
    + '<span class="sq-fx__art" aria-hidden="true">' + artHtml(p) + '</span>'
    + '<div class="sq-fx__head"><span class="sq-fx__grip" aria-hidden="true">' + (kind === 'EFFECT' ?
'⠿' : '⏻') + '</span>'
    + '<span class="sq-fx__name">' + esc(name) + '</span>'
    + (kind === 'EFFECT'
      ? '<button type="button" class="sq-fx__power" aria-pressed="' + String(!d.is_bypassed) + '" aria-l
abel="Bypass">⏻</button>'
      : '')
    + '<button type="button" class="sq-fx__del" aria-label="Remove ' + (kind === 'EFFECT' ? 'effect' :
'instrument') + '">✕</button></div>'
    + '<div class="sq-knobs">' + knobs + '</div>'
    + '<span class="sq-fx__preset">Preset: ' + esc(d.__preset || 'Init') + '</span>'
    + mix + '</li>';
}

/** 악기 카드 아래 줄 — 겹 볼륨 · 팬 · 솔로 */
function instrumentMixHtml(d) {
  const db = Number(d.volume_db || 0);
  return '<div class="sq-fx__mix">'
    + '<span class="sq-irow__vol sq-num" role="slider" tabindex="0" data-live="layer-volume"'
    + ' aria-label="Layer volume" aria-valuemin="-60" aria-valuemax="6" aria-valuenow="' + db + '">'
    + '<span class="sq-irow__db sq-num">' + fmtDb(db) + '</span><i style="--v:' + volPct(db) + '%"></i>
</span>'
    + '<span class="sq-irow__pan" role="slider" tabindex="0" data-live="layer-pan" aria-label="Pan" styl
e="--p:' + panDeg(Number(d.pan || 0)) + 'deg"></span>'
    + '<button type="button" class="sq-irow__solo" aria-pressed="' + String(!!d.is_solo) + '" aria-label
="Solo layer">S</button></div>';
}

/** 이펙트 카드 아래 줄 — 미니 미터 (엔진이 붙기 전까지 흉내) */
function effectExtraHtml() {
  return '<span class="sq-fx__mini" aria-hidden="true"><i style="--m:38%"></i><i style="--m:26%"></i><i
style="--m:12%"></i></span>';
}
```

문법 풀이

식	뜻
`idAttr`	악기와 이펙트가 쓰는 줄 번호 이름이 달라 골라 담습니다
`bypassed ? ' is-bypassed' : ''`	꺼 둔 이펙트는 흐리게 보이게 합니다
`draggable="' + String(kind === 'EFFECT') + '"`	이펙트 카드만 끌어 순서를 바꿉니다
`data-param-key`	노브가 어느 파라미터인지 적어 둡니다
`style="--a:...deg"`	CSS 변수로 노브 각도를 넘깁니다
`(kind === 'EFFECT' ? '...' : '')`	바이패스 버튼은 이펙트 카드에만 넣습니다
`d.__preset || 'Init'`	프리셋 이름은 화면에서만 쓰는 값이라 `__` 를 붙였습니다

4. 왜 이렇게

선택	이유
값을 글자로 만들기	카드가 수십 장이라 문자열로 만드는 편이 빠릅니다
줄 번호를 카드에 적음	저장 · 떼기 요청에 그 줄 번호가 필요합니다
이펙트만 끌 수 있게	악기 겹 차례는 소리 결과를 거의 바꾸지 않습니다 (FR-263)
아트 자리를 늘 남김	그림이 생기면 CSS 만 바꿔 끼울 수 있습니다

5. 시험

시험 프로젝트에서 Bass Synth 칸의 이펙트 카드 2장이 `data-track-plugin-id` 를 각각 42 와 39 로 가지고, 악기 카드가 `data-track-instrument-id` 를 가지는 것을 확인했습니다. 노브 값 글자는 `-20.0 dB`, `0 %` 처럼 단위와 함께 나왔습니다.

JavaScript 254 — 믹서 칸과 아래 믹서 줄 (`mixerHtml` · `renderMixerStrips` · `buildStrip`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 펼침 칸의 믹서와 화면 아래 믹서 줄을 그립니다 (FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	253 Nyxi 카드 한 장 · 255 값 다듬기와 표시 글

1. 하는 일

`mixerHtml` 은 펼친 트랙의 믹서 칸 속을 만듭니다. 볼륨 페이더, 미터, dB 글, 팬, 겹 보내기 자리(A · B)가 담기고 MASTER 는 볼륨만 담습니다. `renderMixerStrips` 는 화면 아래 믹서 줄을 트랙마다 하나씩 다시 그립니다 (MASTER 는 늘 맨 끝). `buildStrip` 은 그 줄 하나를 만듭니다. 줄에는 이름, 악기 이름, `FX 2`, 페이더, dB, 팬, M · S · ● 버튼이 담깁니다.

2. 언제 쓰나

쓰는 곳	
`renderCells` (251)	믹서 칸을 채울 때 `mixerHtml` 을 씁니다
`renderTracks` (246) · `refreshCells` (251) · `syncTrackSubs` (245)	아래 믹서 줄을 다시 그릴 때 `renderMixerStrips` 를 씁니다
`wireControls` (258)	`data-live="track-volume"` 자리를 조작에 묶습니다

3. 코드

```
/** 펼침 칸의 믹서 — 볼륨 · 팬 · 미터 (MASTER 는 볼륨만) */
function mixerHtml(track) {
  const db = Number(track.volume_db || 0);
  const master = track.track_type === 'MASTER';
  return '<div class="sq-mixer">'
    + '<div class="sq-vm">'
    + '<span class="sq-fader" role="slider" tabindex="0" data-live="track-volume" aria-label="Volume"'
    + ' aria-valuemin="-60" aria-valuemax="6" aria-valuenow="' + db + '"><i style="--y:' + faderY(db) +
'%"></i></span>'
    + '<span class="sq-meter"><i style="--m:0%"></i><i style="--m:0%"></i></span>'
    + '</div>'
    + '<span class="sq-db sq-num">' + fmtDb(db) + '</span>'
    + (master
      ? '<span class="sq-pan">L<i></i>R</span>'
      : '<span class="sq-pan">L<i role="slider" tabindex="0" data-live="track-pan" aria-label="Pan" styl
e="--p:' + panDeg(Number(track.pan || 0)) + 'deg"></i>R</span>'
        + '<span class="sq-sends" title="Sends come with the mixer work unit">A<i style="--a:40deg"></i>
B<i style="--a:-10deg"></i></span>')
```

```
    + '</div>';
}

/** 아래 칸 믹서 — 실제 트랙마다 줄 하나, MASTER 는 끝에 */
export function renderMixerStrips() {
  if (!active) return;
  const list = document.getElementById('mixer-strips');
  if (!list) return;
  const ordered = tracks.slice().sort(function (a, b) { return a.order_index - b.order_index; });
  const master = ordered.filter(function (t) { return t.track_type === 'MASTER'; })[0] || null;
  list.innerHTML = '';
  ordered.forEach(function (track) {
    if (track.track_type === 'MASTER') return;
    list.append(buildStrip(track, false));
  });
  const old = document.querySelector('#panel-mixer .sq-strip--master');
  if (master) {
    const strip = buildStrip(master, true);
    if (old) old.replaceWith(strip);
    else list.after(strip);
  }
  const head = document.querySelector('#panel-mixer .sq-bottom__head span');
  if (head) head.textContent = list.children.length + ' tracks · MASTER';
}

/** 아래 칸 믹서 줄 하나 만들기 */
function buildStrip(track, master) {
  const db = Number(track.volume_db || 0);
  const info = stripInfo(track);
  const strip = document.createElement('div');
  strip.className = 'sq-strip is-live' + (master ? ' sq-strip--master' : '');
  strip.dataset.trackId = String(track.track_id);
  strip.dataset.color = track.track_type === 'AUDIO' ? 'audio' : '1';
  strip.innerHTML = '<b class="sq-strip__name">' + esc(info.title) + '</b>'
    + '<small class="sq-strip__sub">' + esc(info.sub) + '</small>'
    + '<span class="sq-strip__fx">FX ' + info.fx + '</span>'
    + '<div class="sq-vm"><span class="sq-fader" role="slider" tabindex="0" data-live="track-volume" ari
a-label="Volume"'
    + ' aria-valuemin="-60" aria-valuemax="6" aria-valuenow="' + db + '"><i style="--y:' + faderY(db) +
'%"></i></span>'
    + '<span class="sq-meter"><i style="--m:0%"></i><i style="--m:0%"></i></span></div>'
    + '<span class="sq-db sq-num">' + fmtDb(db) + '</span>'
    + '<span class="sq-pan">L<i' + (master ? '' : ' role="slider" tabindex="0" data-live="track-pan" ari
a-label="Pan" style="--p:' + panDeg(Number(track.pan || 0)) + 'deg"') + '></i>R</span>'
    + '<span class="sq-msr"><button type="button" class="is-mute" aria-pressed="' + String(!!track.is_mu
ted) + '" aria-label="Mute">M</button>'
    + (master ? '' : '<button type="button" class="is-solo" aria-pressed="' + String(!!track.is_solo) +
'" aria-label="Solo">S</button>'
      + '<button type="button" class="is-arm" aria-pressed="false" aria-label="Record arm">●</button>')
    + '</span>';
  return strip;
}
```

문법 풀이

식	뜻
`master ? ... : ...`	MASTER 는 팬과 보내기 자리를 만들지 않습니다
`data-live="track-volume"`	조작을 묶을 자리를 표시하는 이름입니다
`if (!active) return;`	데모 모드에서는 아래 믹서 줄을 건드리지 않습니다
`list.innerHTML = ''`	이전 줄을 모두 지웁니다
`old.replaceWith(strip)`	이미 있던 MASTER 줄 자리에 새 줄을 갈아 끼웁니다
`list.children.length + ' tracks · MASTER'`	머리 글에 트랙 수를 적습니다

4. 왜 이렇게

선택	이유
MASTER 를 끝에	화면 자리가 고정되어 있어 늘 맨 끝이어야 합니다
MASTER 만 따로 갈아 끼움	MASTER 줄은 목록 밖에 있어 지웠다 만들면 자리가 흔들립니다
미터를 0% 로 시작	소리 값은 다음 작업이라 지금은 자리만 잡아 둡니다
`is-live` 를 붙임	예시 믹서 줄과 구분해 조작이 겹치지 않게 합니다

5. 시험

시험 프로젝트에서 아래 믹서 줄이 5줄(MASTER 제외)과 MASTER 줄 1개로 그려지고, 머리 글이 `5 tracks · MASTER` 인 것을 확인했습니다. Bass Synth 줄에서 페이더를 위로 끌어 `-20.0 dB` 에서 `0.0 dB` 로 바뀐 뒤 `GET /tracks/4639` 의 `volume_db` 가 함께 바뀐 것을 확인했습니다.

JavaScript 255 — 값 다듬기와 표시 글 (`clampValue` · `normOf` · `valueOfNorm` · `knobAngle` · `fmtValue` · `fmtDb` · `volPct` · `faderY` · `panDeg` · `numOr` · `selectLabel`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 파라미터 값을 범위에 맞추고 화면에 쓸 글과 자리를 만듭니다 (FR-244 · FR-2187)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	254 믹서 칸과 아래 믹서 줄 · 256 파라미터 기본값

1. 하는 일

노브와 페이더는 값을 그대로 쓰지 않고 세 단계를 거칩니다. 첫째, 값이 범위를 벗어나지 않게 다듬고(`clampValue`) step 에 맞춥니다. 둘째, 노브를 돌리기 쉽도록 값을 0~1 자리로 바꾸거나(`normOf`) 되돌립니다(`valueOfNorm`). 셋째, 화면에 쓸 글을 만듭니다(`fmtValue`). 소리가 로그 곡선을 쓰는 파라미터는 같은 자리 이동에도 값이 곱셈으로 바뀝니다. 나머지 함수들은 dB 글, 막대 길이, 손잡이 위치, 팬 각도처럼 화면 자리를 만드는 작은 도우미입니다.

함수	하는 일	쓰는 곳
`clampValue`	범위 안으로 자르고 step 에 맞춤	모든 값 저장
`normOf`	값 → 0~1 자리	끌기 시작, 노브 각도
`valueOfNorm`	0~1 자리 → 값	끌기, 휠, 키
`knobAngle`	값 → 노브 각도	카드 노브
`fmtValue`	값 → 글 (`-20.0 dB`, `On`, `Low`)	노브 값, 안내
`fmtDb`	dB 글 (`+3.0 dB`)	페이더, 겹 볼륨
`volPct`	dB → 막대 길이 %	겹 볼륨 막대
`faderY`	dB → 손잡이 위치 %	세로 페이더
`panDeg`	팬(-1~1) → 각도	팬 노브
`numOr`	값이 없으면 대체 값	카드 aria

2. 언제 쓰나

쓰는 곳	
`setControlValue` (257) · `onControlMove` (257)	값을 다듬고 자리를 오갈 때 씁니다
`cardHtml` (253) · `paintKnob` (258) · `buildStrip` (254)	글과 자리를 만들 때 씁니다

3. 코드

```
/** 값 범위 안으로 다듬고 step 에 맞추기 (step 0 = 자유) */
export function clampValue(item, v) {
  const min = item && item.min != null ? item.min : 0;
```

```
  const max = item && item.max != null ? item.max : 1;
  let out = Math.min(max, Math.max(min, Number(v)));
  const step = item && item.step ? item.step : 0;
  if (step > 0) out = Math.round(out / step) * step;
  return Number(out.toFixed(4)) || 0;
}

/** 값 → 0~1 자리 (log 곡선이면 로그 자리) */
export function normOf(item, v) {
  const min = item && item.min != null ? item.min : 0;
  const max = item && item.max != null ? item.max : 1;
  if (item && item.curve === 'log' && min > 0 && max > min) {
    return Math.log(Number(v) / min) / Math.log(max / min);
  }
  return (Number(v) - min) / ((max - min) || 1);
}

/** 0~1 자리 → 값 */
export function valueOfNorm(item, n) {
  const min = item && item.min != null ? item.min : 0;
  const max = item && item.max != null ? item.max : 1;
  const k = Math.min(1, Math.max(0, n));
  if (item && item.curve === 'log' && min > 0 && max > min) {
    return clampValue(item, min * Math.pow(max / min, k));
  }
  return clampValue(item, min + k * (max - min));
}

/** 값 → 노브 각도 (가운데 0, 좌우 KNOB_SWEEP) */
export function knobAngle(item, v) {
  return Math.round((normOf(item, v) * 2 - 1) * KNOB_SWEEP);
}

/** 값 → 화면에 쓸 글 (단위 · 소수 자리) */
export function fmtValue(item, v) {
  if (!item) return String(v);
  if (item.type === 'toggle') return v ? 'On' : 'Off';
  if (item.type === 'select') return selectLabel(item, v);
  const step = item.step ? item.step : 0;
  const dec = step >= 1 ? 0 : (step >= 0.1 ? 1 : (step >= 0.01 ? 2 : 3));
  const unit = item.unit ? (item.unit === '%' ? ' %' : ' ' + item.unit) : '';
  return Number(v).toFixed(dec) + unit;
}

/** dB 글자 */
function fmtDb(db) {
  const v = Number(db) || 0;
  return (v > 0 ? '+' : '') + v.toFixed(1) + ' dB';
}

/** 겹 볼륨 막대 길이(%) — -60 dB = 0%, +6 dB = 100% */
function volPct(db) {
  return Math.round(((Math.min(6, Math.max(-60, Number(db) || 0)) + 60) / 66) * 100);
}

/** 페이더 손잡이 위치(%) — +6 dB = 0%(위), -60 dB = 100%(아래) */
function faderY(db) {
```

```
  return Math.round(((6 - Math.min(6, Math.max(-60, Number(db) || 0))) / 66) * 100);
}

/** 팬(-1 ~ 1) → 각도 */
function panDeg(v) {
  return Math.round(Math.min(1, Math.max(-1, Number(v) || 0)) * KNOB_SWEEP);
}

/** 값에서 쓸 수 있는 숫자 */
function numOr(v, fallback) {
  return v == null ? fallback : Number(v);
}
```

문법 풀이

식	뜻
`item.min != null ? item.min : 0`	값이 없을 때와 0 일 때를 갈라 봅니다
`Math.min(max, Math.max(min, ...))`	위아래 한계 안으로 가둡니다
`Math.round(out / step) * step`	step 배수로 맞춥니다
`Number(out.toFixed(4))`	부동소수점 찌꺼기를 지웁니다
`item.curve === 'log'`	로그 곡선 파라미터인지 봅니다
`Math.log(v / min) / Math.log(max / min)`	로그 자리로 바꿉니다
`min * Math.pow(max / min, k)`	로그 자리에서 값으로 되돌립니다
`normOf(...) * 2 - 1`	0~1 을 -1~1 로 바꿔 좌우 각도를 만듭니다
`toFixed(dec)`	step 크기에 따라 소수 자리를 정합니다
`item.unit === '%'`	퍼센트는 앞에 띄어쓰기를 넣습니다

4. 왜 이렇게

선택	이유
자리(0~1)를 거쳐 바꿈	끌기 · 휠 · 키가 모두 같은 계산을 쓰게 됩니다
로그 곡선	소리 크기와 주파수는 사람이 로그로 느낍니다
자유 파라미터는 step 0	step 이 없으면 값을 그대로 둡니다
`toFixed(4)`	0.1 을 더할 때 생기는 `0.30000000000000004` 를 막습니다
dB 표시를 따로	페이더 눈금은 파라미터 스키마 밖의 고정 범위(-60 ~ +6)입니다

5. 시험

노브 드래그 시험에서 `aria-valuenow` 가 `-20` 에서 `0` 으로 바뀌고 값 글자가 `-20.0 dB` 로 함께 바뀌는 것을 확인했습니다. dB 를 +6 으로 두면 페이더 손잡이 위치가 0%(맨 위), -60 으로 두면 100%(맨 아래)가 되는 것도 확인했습니다.

JavaScript 256 — 파라미터 기본값 (`defaultParams` · `withDefaults` · `schemaItem`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 플러그인 기본값으로 파라미터를 온전하게 만듭니다 (FR-244)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	255 값 다듬기와 표시 글 · 257 조작 연결

1. 하는 일

플러그인 표에는 파라미터마다 기본값이 담긴 스키마가 있습니다. `defaultParams` 는 그 기본값만 모아 params 를 만들고, `withDefaults` 는 서버에 저장된 값 위에 기본값을 깔아 **빠진 키가 없는** params 를 만듭니다. 그래서 나중에 플러그인에 파라미터가 하나 늘어도 저장된 옛 값이 있어 그대로 동작합니다. `schemaItem` 은 키로 스키마 항목 하나를 찾습니다.

```
defaultParams(plugin)          { cutoff: 800, reso: 0.3, ... }   기본값만
withDefaults(plugin, saved)    기본값 위에 saved 를 덮음        빠진 키 없음
```

2. 언제 쓰나

쓰는 곳	
`addDevice` (260)	새 카드를 붙일 때 `defaultParams` 를 보냅니다
`cardHtml` (253) · `wireControls` (258)	저장 값에 기본값을 깔아 노브 값을 만듭니다
`applyPreset` (261) · `saveDeviceModal` (261)	프리셋과 저장 뒤 params 를 다시 맞춥니다
`wireControls` (258)	`schemaItem` 으로 노브에 맞는 스키마를 찾습니다

3. 코드

```
/** 플러그인의 기본값만 모은 params (붙일 때 · 초기화할 때) */
export function defaultParams(plugin) {
  const out = {};
  const schema = (plugin && plugin.param_schema) || [];
  schema.forEach(function (item) { out[item.key] = item.default; });
  return out;
}

/** 저장된 params 위에 기본값을 깔아 완전한 params 만들기 (빠진 키 채우기) */
export function withDefaults(plugin, saved) {
  const out = defaultParams(plugin);
  if (saved) {
    Object.keys(saved).forEach(function (k) { out[k] = saved[k]; });
  }
```

```
  return out;
}

/** 스키마에서 키로 항목 찾기 */
function schemaItem(plugin, key) {
  const schema = (plugin && plugin.param_schema) || [];
  for (let i = 0; i < schema.length; i += 1) {
    if (schema[i].key === key) return schema[i];
  }
  return null;
}
```

문법 풀이

식	뜻
`const schema = (plugin && plugin.param_schema) || [];`	플러그인이 없거나 스키마가 비어도 빈 배열로 시작합니다
`out[item.key] = item.default`	키 이름 그대로 값을 담습니다
`defaultParams(plugin)`	기본값을 먼저 깔고
`Object.keys(saved).forEach(...)`	저장 값이 있으면 그 위에 덮습니다
`schema[i].key === key`	스키마 항목의 키와 찾는 키를 맞춰 봅니다

4. 왜 이렇게

선택	이유
기본값을 먼저 깔기	옛 카드에 새 파라미터가 없어도 노브가 빈 채로 뜨지 않습니다
사본을 만들어 반환	원본 저장 값(장치 객체)을 건드리지 않습니다
초기화에도 같은 함수	Init 버튼이 붙일 때와 같은 값으로 돌아갑니다
`schemaItem` 을 따로	같은 스키마 찾기를 여러 곳에서 씁니다

5. 시험

시험 프로젝트에서 EQ3 를 Init 으로 되돌리면 Low · Mid · High 세 값이 모두 스키마의 `default` 로 돌아오는 것을 확인했습니다. 저장 값에 `low` 하나만 넣고(다른 키 없이) 카드를 그려도 나머지 노브가 기본값으로 채워지는 것도 확인했습니다.

JavaScript 257 — 조작 연결 (`bindControl` · `setControlValue` · `syncAria` · `onControlDown` ~ `onControlKey`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 노브 · 페이더 · 팬을 마우스 · 휠 · 키보드로 다룹니다 (FR-2187)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	256 파라미터 기본값 · 258 칸 안 조작 연결

1. 하는 일

`bindControl` 은 화면 요소 하나에 조작을 묶습니다. 위로 끌면 값이 커지고, Shift 를 함께 누르면 10배 미세 조정이 됩니다. 휠 한 칸은 step 만큼, 화살표 키와 PageUp · PageDown, Home · End 도 값을 바꿉니다.
`setControlValue` 는 새 값을 다듬어 화면 글과 화면 낭독기 값을 함께 맞추고, `commit=true` 이면 서버에 저장합니다. `onControlDown` 부터 다섯 함수는 실제 입력을 받아 그 계산을 부릅니다.

2. 언제 쓰나

쓰는 곳	
`wireControls` (258)	칸 안의 모든 조작 자리에 `bindControl` 을 답니다
`bindModalParam` (261)	상세 모달의 노브에도 같은 조작을 답니다
`wireDemoControls` (262)	예시 화면 노브에 조작만 붙입니다(저장 없음)

3. 코드

```
export function bindControl(el, cfg) {
  if (!el || el.dataset.devBound === '1') return;
  el.dataset.devBound = '1';
  el.devCfg = cfg;
  if (!el.hasAttribute('tabindex')) el.setAttribute('tabindex', '0');
  if (!el.hasAttribute('role')) el.setAttribute('role', 'slider');
  el.addEventListener('pointerdown', onControlDown);
  el.addEventListener('pointermove', onControlMove);
  el.addEventListener('pointerup', onControlUp);
  el.addEventListener('pointercancel', onControlUp);
  el.addEventListener('wheel', onControlWheel, { passive: false });
  el.addEventListener('keydown', onControlKey);
  syncAria(el, cfg);
}

/** 조작 하나의 값을 바꾸고 화면 · aria 를 맞춤 (commit=true 면 서버 저장까지) */
export function setControlValue(el, cfg, v, commit) {
  const item = cfg.item || cfg;
  cfg.value = clampValue(item, v);
```

```
  if (cfg.render) cfg.render(cfg.value);
  syncAria(el, cfg);
  if (commit && cfg.commit) cfg.commit(cfg.value);
}

/** aria-valuenow · aria-valuetext 맞추기 */
function syncAria(el, cfg) {
  el.setAttribute('aria-valuenow', String(cfg.value));
  const text = cfg.type === 'select' ? cfg.value : fmtValue(cfg.item || cfg, cfg.value);
  el.setAttribute('aria-valuetext', String(text));
}

function onControlDown(e) {
  const cfg = e.currentTarget.devCfg;
  if (!cfg || e.button !== 0) return;
  e.preventDefault();
  dragging = {
    el: e.currentTarget,
    startY: e.clientY,
    startNorm: normOf(cfg.item || cfg, cfg.value),
  };
  e.currentTarget.classList.add('is-drag');
  if (e.currentTarget.setPointerCapture) e.currentTarget.setPointerCapture(e.pointerId);
}

function onControlMove(e) {
  if (!dragging || dragging.el !== e.currentTarget) return;
  const cfg = e.currentTarget.devCfg;
  const item = cfg.item || cfg;
  const fine = e.shiftKey ? 0.1 : 1;
  const dn = ((dragging.startY - e.clientY) * fine) / (cfg.pixels || DRAG_PIXELS);
  setControlValue(e.currentTarget, cfg, valueOfNorm(item, dragging.startNorm + dn), false);
}

function onControlUp(e) {
  if (!dragging || dragging.el !== e.currentTarget) return;
  const cfg = e.currentTarget.devCfg;
  e.currentTarget.classList.remove('is-drag');
  dragging = null;
  if (cfg && cfg.commit) cfg.commit(cfg.value);
}

function onControlWheel(e) {
  const cfg = e.currentTarget.devCfg;
  if (!cfg) return;
  const item = cfg.item || cfg;
  e.preventDefault();
  const step = item.step > 0 ? item.step : (item.max - item.min) / 100;
  const dir = e.deltaY < 0 ? 1 : -1;
  const fine = e.shiftKey ? 0.1 : 1;
  setControlValue(e.currentTarget, cfg, cfg.value + dir * step * fine, true);
}

function onControlKey(e) {
  const cfg = e.currentTarget.devCfg;
  if (!cfg) return;
  const item = cfg.item || cfg;
```

```
  const step = item.step > 0 ? item.step : (item.max - item.min) / 100;
  let v = null;
  if (e.key === 'ArrowUp' || e.key === 'ArrowRight') v = cfg.value + step;
  else if (e.key === 'ArrowDown' || e.key === 'ArrowLeft') v = cfg.value - step;
  else if (e.key === 'PageUp') v = cfg.value + step * 10;
  else if (e.key === 'PageDown') v = cfg.value - step * 10;
  else if (e.key === 'Home') v = item.min;
  else if (e.key === 'End') v = item.max;
  if (v === null) return;
  e.preventDefault();
  setControlValue(e.currentTarget, cfg, v, true);
}
```

문법 풀이

식	뜻
`el.dataset.devBound === '1'`	이미 묶인 자리는 두 번 묶지 않습니다
`el.devCfg = cfg`	요소에 조작 설정을 담아 두고 입력 때 꺼내 씁니다
`{ passive: false }`	휠에서 기본 스크롤을 막으려면 필요합니다
`e.currentTarget.devCfg`	지금 다루는 요소의 설정입니다
`e.button !== 0`	마우스 왼쪽 버튼만 받습니다
`setPointerCapture`	끌기가 요소 밖으로 나가도 계속 받습니다
`(startY - clientY)`	위로 끌면 양수가 되어 값이 커집니다
`e.shiftKey ? 0.1 : 1`	Shift 로 미세 조정합니다
끌기는 `commit=false`	다 놓을 때(`onControlUp`) 한 번만 저장합니다
휠 · 키는 `commit=true`	한 번에 한 칸이라 바로 저장해도 요청이 많지 않습니다

4. 왜 이렇게

선택	이유
한 함수로 모든 조작	노브 · 페이더 · 팬이 같은 손맛을 갖습니다
자리 계산으로 처리	값 범위가 파라미터마다 달라도 같은 코드로 다룹니다
끌 때는 저장 안 함	끄는 동안 매번 저장하면 요청이 쏟아집니다
설정을 요소에 담음	함수마다 상태를 따로 넘기지 않아도 됩니다

5. 시험

Bass Synth 카드의 노브를 위로 끌어 `aria-valuenow` 가 `-20` 에서 `0` 으로 바뀌고, 놓는 순간에만 `PATCH` 가 나가는 것을 확인했습니다(끄는 동안 요청 0건, 놓은 뒤 1건). 화살표 키로 한 칸씩, Shift 와 함께 끌면 10배 천천히 움직이는 것도 확인했습니다.

JavaScript 258 — 칸 안 조작 연결 (`wireControls` · `deviceOfCard` · `paintKnob`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 칸 안의 노브 · 페이더 · 팬을 값과 이어 줍니다 (FR-2187 · FR-245)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	257 조작 연결 · 259 플러그인 고르기 창

1. 하는 일

`wireControls` 는 펼침 칸 하나를 훑어 조작 자리마다 `bindControl` 을 답니다. 트랙 볼륨과 팬, 카드 노브, 겹 볼륨과 팬 네 가지입니다. 각 자리마다 값이 어디서 오고(트랙 값인지 장치 값인지) 저장할 때 어느 함수를 부를지 정해 줍니다. `deviceOfCard` 는 카드에 적힌 줄 번호로 담아 둔 장치를 찾고, `paintKnob` 은 노브의 각도와 값 글자를 그립니다.

2. 언제 쓰나

쓰는 곳	
`renderCells` (251)	칸을 그린 뒤 마지막에 부릅니다
`bindModalParam` (261)	모달 노브는 `paintKnob` 을 그대로 씁니다
`toggleBypass` · `toggleSolo` (262) · `wireCells` (263)	카드의 장치를 찾을 때 `deviceOfCard` 를 씁니다

3. 코드

```
export function wireControls(scope, track) {
  Array.prototype.forEach.call(scope.querySelectorAll('[data-live="track-volume"]'), function (el) {
    const box = el.closest('.sq-strip') || el.closest('.sq-cell');
    bindControl(el, {
      item: { min: -60, max: 6, step: 0.1 },
      value: Number(track.volume_db) || 0,
      pixels: 120,
      render: function (v) {
        const bar = el.querySelector('i');
        if (bar) bar.style.setProperty('--y', faderY(v) + '%');
        const text = box && box.querySelector('.sq-db');
        if (text) text.textContent = fmtDb(v);
      },
      commit: function (v) {
        track.volume_db = v;
        updateTrack(track, { volume_db: v }).catch(warn);
      },
    });
  });
```

```
Array.prototype.forEach.call(scope.querySelectorAll('[data-live="track-pan"]'), function (el) {
  bindControl(el, {
    item: { min: -1, max: 1, step: 0.01 },
    value: Number(track.pan) || 0,
    pixels: 100,
    render: function (v) { el.style.setProperty('--p', panDeg(v) + 'deg'); },
    commit: function (v) {
      track.pan = v;
      updateTrack(track, { pan: v }).catch(warn);
    },
  });
});

Array.prototype.forEach.call(scope.querySelectorAll('.sq-fx[data-device-kind]'), function (card) {
  const device = deviceOfCard(track, card);
  const plugin = device ? pluginById(device.plugin_id) : null;
  if (!device || !plugin) return;
  const values = withDefaults(plugin, device.params);

  Array.prototype.forEach.call(card.querySelectorAll('.sq-knob[data-param-key]'), function (el) {
    const item = schemaItem(plugin, el.dataset.paramKey + '');
    if (!item) return;
    bindControl(el, {
      item: item,
      value: values[item.key],
      pixels: 130,
      render: function (v) { paintKnob(el, item, v); },
      commit: function (v) {
        const next = withDefaults(plugin, device.params);
        next[item.key] = v;
        device.params = next;
        updateDevice(track, card.dataset.deviceKind, device, { params: next }).catch(warn);
      },
    });
  });

  Array.prototype.forEach.call(card.querySelectorAll('[data-live="layer-volume"]'), function (el) {
    bindControl(el, {
      item: { min: -60, max: 6, step: 0.1 },
      value: Number(device.volume_db) || 0,
      pixels: 100,
      render: function (v) {
        const bar = el.querySelector('i');
        if (bar) bar.style.setProperty('--v', volPct(v) + '%');
        const out = el.querySelector('.sq-irow__db');
        if (out) out.textContent = fmtDb(v);
      },
      commit: function (v) {
        device.volume_db = v;
        updateDevice(track, card.dataset.deviceKind, device, { volume_db: v }).catch(warn);
      },
    });
  });

  Array.prototype.forEach.call(card.querySelectorAll('[data-live="layer-pan"]'), function (el) {
    bindControl(el, {
```

```
        item: { min: -1, max: 1, step: 0.01 },
        value: Number(device.pan) || 0,
        pixels: 80,
        render: function (v) { el.style.setProperty('--p', panDeg(v) + 'deg'); },
        commit: function (v) {
          device.pan = v;
          updateDevice(track, card.dataset.deviceKind, device, { pan: v }).catch(warn);
        },
      });
    });
  });
}

/** 카드의 장치 찾기 (devices 캐시에서) */
function deviceOfCard(track, card) {
  const d = devices[track.track_id];
  if (!d) return null;
  if (card.dataset.trackPluginId) {
    const id = Number(card.dataset.trackPluginId);
    return d.effects.filter(function (x) { return x.track_plugin_id === id; })[0] || null;
  }
  const id = Number(card.dataset.trackInstrumentId);
  return d.instruments.filter(function (x) { return x.track_instrument_id === id; })[0] || null;
}

/** 노브 그리기 — 각도와 값 글자 */
export function paintKnob(el, item, v) {
  const ring = el.querySelector('i');
  if (ring) ring.style.setProperty('--a', knobAngle(item, v) + 'deg');
  const out = el.querySelector('.sq-knob__val');
  if (out) out.textContent = fmtValue(item, v);
}
```

문법 풀이

식	뜻
`scope.querySelectorAll('[data-live="track-volume"]')`	칸 안의 볼륨 자리를 모두 찾습니다
`el.closest('.sq-strip') || el.closest('.sq-cell')`	dB 글을 어디서 찾을지 정합니다(믹서 줄인지 칸인지)
`item: { min: -60, max: 6, step: 0.1 }`	트랙 값은 스키마가 없어 범위를 직접 적습니다
`scope.querySelectorAll('.sq-fx[data-device-kind]')`	카드마다 한 번씩 돕니다
`el.dataset.paramKey + ''`	노브에 적힌 파라미터 키를 글자로 꺼냅니다
`withDefaults(plugin, device.params)`	저장 값에 기본값을 깔아 완전한 값으로 만듭니다
`next[item.key] = v`	사본에 새 값만 넣고 통째로 보냅니다
`deviceOfCard`	카드의 줄 번호로 장치를 찾습니다

4. 왜 이렇게

선택	이유
값 → 저장 → 캐시 갱신	담아 둔 장치를 고쳐 두어야 다음 그리기에서 값이 맞습니다

선택	이유
칸마다 다시 묶음	값이 바뀌면 칸을 다시 그리므로 묶음도 새로 필요합니다
`devBound` 로 한 번만	다시 그려도 같은 요소에 두 번 묶이지 않게 `bindControl` 이 막습니다
카드 사본을 통째로 저장	서버가 params 를 통째로 받으므로 빠진 키 없이 보내야 합니다

5. 시험

Bass Synth 카드 노브를 끌어 값이 바뀌고, 놓은 뒤 `GET /tracks/4639/plugins` 의 해당 줄 `params` 가 바뀐 것을 확인했습니다. 아래 믹서 줄 페이더와 펼침 칸 페이더가 같은 트랙 값을 함께 보고 있는 것도 확인했습니다.

JavaScript 259 — 플러그인 고르기 창 (`openPluginPicker` · `renderPickerList` · `paramSummary`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 붙일 플러그인을 고르는 창을 엽니다 (FR-244)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	258 칸 안 조작 연결 · 260 붙이기 · 고치기 · 떼기 · 순서

1. 하는 일

카드 끝의 `+` 를 누르면 `openPluginPicker` 가 열립니다. 악기를 붙이는 중이면 악기만, 이펙트를 붙이는 중이면 이펙트만 보여 줍니다. `renderPickerList` 는 목록을 그리고 검색어로 이름과 코드를 걸러 줍니다. `paramSummary` 는 플러그인마다 `"3 knobs · 3 on the card"` 처럼 노브 수를 알려 줍니다. 고른 플러그인은 넘겨받은 `onPick` 함수로 전달됩니다.

```
+ → openPluginPicker(kind, onPick)
      ├ renderPickerList('')        처음 목록
      ├ 검색어 입력 → renderPickerList(검색어)
      └ 목록에서 고름 → onPick(plugin) → addDevice (260)
```

2. 언제 쓰나

쓰는 곳	
`wireCells` (263)	`+` 클릭에서 `openPluginPicker` 를 부릅니다
`wireDialogs` (263)	검색어 입력과 목록 클릭을 배선합니다

3. 코드

```
export function openPluginPicker(kind, onPick) {
  if (!active || !plugins.length) {
    showToast('Open a project with ?projectId=… to add plugins — this is the example screen');
    return;
  }
  picker = { kind: kind, onPick: onPick };
  const dlg = document.getElementById('plugin-picker');
  const title = document.getElementById('plugin-picker-title');
  const note = document.getElementById('picker-note');
  const search = document.getElementById('picker-search');
  if (title) title.textContent = kind === 'INSTRUMENT' ? 'Choose an instrument' : 'Choose an effect';
  if (note) note.textContent = plugins.length + ' plugins in the Plugins table (FR-244)';
  if (search) search.value = '';
  renderPickerList('');
  if (dlg && dlg.showModal) dlg.showModal();
```

```
  if (search) search.focus();
}

/** 고르기 창 목록 그리기 (검색어로 거르기) */
export function renderPickerList(query) {
  const list = document.getElementById('picker-list');
  if (!list || !picker) return;
  const want = picker.kind === 'INSTRUMENT' ? 'INSTRUMENT' : 'EFFECT';
  const q = String(query || '').trim().toLowerCase();
  const found = plugins.filter(function (p) {
    if (p.plugin_type !== want) return false;
    if (!q) return true;
    return p.name.toLowerCase().indexOf(q) >= 0 || String(p.code).toLowerCase().indexOf(q) >= 0;
  });
  if (!found.length) {
    list.innerHTML = '<li class="sq-picker__empty">No plugin matches. Plugins are added to the Plugins t
able by the admin screen.</li>';
    return;
  }
  list.innerHTML = found.map(function (p) {
    return '<li><button type="button" class="sq-plug" data-plugin-id="' + p.plugin_id + '">'
      + '<span class="sq-plug__art" aria-hidden="true">' + artHtml(p) + '</span>'
      + '<span><span class="sq-plug__name">' + esc(p.name) + '</span><br>'
      + '<span class="sq-plug__sub">' + esc(p.code) + ' · ' + esc(p.engine) + ' · v' + esc(p.version) +
' . '
      + paramSummary(p) + '</span></span>'
      + '<span class="sq-plug__badge">' + esc(p.plugin_type) + '</span></button></li>';
  }).join('');
}

/** 플러그인 파라미터 요약 — "47 parameters · 18 on the card" 처럼 */
function paramSummary(plugin) {
  const schema = plugin.param_schema || [];
  const knobs = schema.filter(function (it) { return it.type === 'knob'; }).length;
  return knobs + ' knobs · ' + primaryItems(plugin).length + ' on the card';
}
```

문법 풀이

식	뜻
`!active || !plugins.length`	데모 모드이거나 표가 비었으면 안내만 띄웁니다
`picker = { kind, onPick }`	무엇을 고르는 중인지 담아 둡니다
`dlg.showModal()`	브라우저 기본 대화 상자로 엽니다
`search.focus()`	열자마자 검색창에 커서를 둡니다
`p.name.toLowerCase().indexOf(q) >= 0`	이름이나 코드 안에 검색어가 있는지 봅니다
`p.plugin_type !== want`	붙이는 종류와 다른 플러그인은 걸러 냅니다
`data-plugin-id`	고른 플러그인을 찾을 번호입니다

4. 왜 이렇게

선택	이유
종류로 거르기	악기 자리에 이펙트를 붙이면 서버가 거절합니다
검색어로 이름과 코드	코드로도 찾으면 플러그인이 많아져도 빠릅니다
`onPick` 을 넘김	같은 창을 악기와 이펙트 양쪽에서 씁니다
표가 비면 안내	관리 화면에서 늘릴 수 있다는 것을 알려 줍니다

5. 시험

Bass Synth 의 `+ Add instrument` 를 눌렀을 때 창 제목이 `Choose an instrument` 이고 악기 2종만 나오는 것을 확인했습니다. 이펙트 `+` 로 열면 `Choose an effect` 와 5종이 나왔습니다. `com` 을 입력하면 `Compressor` 한 줄만 남았습니다.

JavaScript 260 — 붙이기 · 고치기 · 떼기 · 순서 (`addDevice` · `tailSlot` · `updateDevice` · `removeDevice` · `moveEffect` · `byId`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 장치를 붙이고 고치고 떼고 순서를 바꿉니다 (FR-244 · FR-245 · FR-263)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	259 플러그인 고르기 창 · 261 상세 설정 모달

1. 하는 일

네 함수가 서버의 장치 줄을 다룹니다. `addDevice` 는 고른 플러그인을 트랙에 붙이고(악기는 instruments, 이펙트는 plugins) 기본값으로 시작합니다. `tailSlot` 은 붙일 자리를 정합니다. `updateDevice` 는 값 · 바이패스 · 솔로 · 볼륨 · 자리 하나를 고칩니다. `removeDevice` 는 확인을 받고 뗍니다. `moveEffect` 는 이펙트 차례를 바꾸는데, 이펙트 칸 번호에는 `UQ(track_id, slot_index)` 라는 중복 금지 규칙이 있어 옮길 카드를 빈 칸 100 에 잠깐 두고 사이 카드를 한 칸씩 밀거나 당깁니다.

```
moveEffect(track, 1, 0)
  ① [A][B] 에서 B 를 slot 100 으로 비켜 둠   →  A(0), B(100)
  ② 사이 카드를 한 칸씩 당김                →  A(1), B(100)
  ③ B 를 갈 자리로                          →  A(1), B(0)
```

2. 언제 쓰나

쓰는 곳	
`wireCells` (263) · 고르기 창 콜백	`addDevice` · `removeDevice` 를 부릅니다
`wireChainDrag` (263)	끌어 놓은 자리로 `moveEffect` 를 부릅니다
`wireControls` (258) · `saveDeviceModal` (261) · `toggleBypass` (262)	`updateDevice` 를 부릅니다

3. 코드

```
export async function addDevice(track, kind, plugin) {
  const d = devices[track.track_id] || { instruments: [], effects: [] };
  devices[track.track_id] = d;
  const list = kind === 'INSTRUMENT' ? d.instruments : d.effects;
  const body = { plugin_id: plugin.plugin_id, slot_index: tailSlot(list), params: defaultParams(plugin)
};
  try {
    const path = kind === 'INSTRUMENT'
      ? '/tracks/' + track.track_id + '/instruments'
      : '/tracks/' + track.track_id + '/plugins';
    const saved = await api.post(path, body);
    list.push(saved);
```

```
    refreshCells(track);
    showToast((kind === 'INSTRUMENT' ? 'Instrument' : 'Effect') + ' added: ' + plugin.name);
  } catch (err) {
    warn(err);
  }
}

/** 새 장치를 붙일 자리 (FR-245) — 지금 것들 뒤(가장 큰 slot_index + 1)가 첫 후보입니다. */
function tailSlot(list) {
  const used = {};
  let top = -1;
  list.forEach(function (x) {
    used[x.slot_index] = true;
    if (x.slot_index > top) top = x.slot_index;
  });
  if (top + 1 <= 126 && !used[top + 1]) return top + 1;
  let n = 0;
  while (used[n] && n < 127) n += 1;
  return n;
}

export async function updateDevice(track, kind, device, patch) {
  try {
    const path = kind === 'INSTRUMENT'
      ? '/tracks/' + track.track_id + '/instruments/' + device.track_instrument_id
      : '/tracks/' + track.track_id + '/plugins/' + device.track_plugin_id;
    const saved = await api.patch(path, patch);
    Object.assign(device, saved);
    return saved;
  } catch (err) {
    warn(err);
    return null;
  }
}

export async function removeDevice(track, kind, device) {
  const plugin = pluginById(device.plugin_id);
  const name = plugin ? plugin.name : 'this device';
  if (window.confirm && !window.confirm('Remove ' + name + '?')) return;
  try {
    const path = kind === 'INSTRUMENT'
      ? '/tracks/' + track.track_id + '/instruments/' + device.track_instrument_id
      : '/tracks/' + track.track_id + '/plugins/' + device.track_plugin_id;
    await api.delete(path);
    const d = devices[track.track_id];
    if (d) {
      const list = kind === 'INSTRUMENT' ? d.instruments : d.effects;
      const at = list.indexOf(device);
      if (at >= 0) list.splice(at, 1);
    }
    refreshCells(track);
    if (modal && modal.device === device) closeDeviceModal();
    showToast(name + ' removed');
  } catch (err) {
    warn(err);
  }
}
```

```
export async function moveEffect(track, from, to) {
  const d = devices[track.track_id];
  if (!d) return;
  const list = sorted(d.effects);
  if (from === to || from < 0 || to < 0 || from >= list.length || to >= list.length) return;
  const ids = list.map(function (x) { return x.track_plugin_id; });
  const movedId = ids.splice(from, 1)[0];
  ids.splice(to, 0, movedId);
  try {
    await updateDevice(track, 'EFFECT', list[from], { slot_index: OPEN_SLOT });
    if (to > from) {
      for (let i = from + 1; i <= to; i += 1) {
        await updateDevice(track, 'EFFECT', byId(list, ids[i]), { slot_index: i - 1 });
      }
    } else {
      for (let i = from - 1; i >= to; i -= 1) {
        await updateDevice(track, 'EFFECT', byId(list, ids[i]), { slot_index: i + 1 });
      }
    }
    await updateDevice(track, 'EFFECT', byId(list, movedId), { slot_index: to });
    await loadDevices(track);
    refreshCells(track);
    showToast('Effect order changed');
  } catch (err) {
    warn(err);
    await loadDevices(track).catch(function () {});
    refreshCells(track);
  }
}

/** 목록에서 id 로 찾기 */
function byId(list, id) {
  return list.filter(function (x) { return x.track_plugin_id === id; })[0];
}
```

문법 풀이

식	뜻
`devices[track.track_id] = d`	값이 없으면 빈 자리를 먼저 만들어 둡니다
`tailSlot(list)`	지금 것들 뒤 빈 칸 번호를 찾습니다
`list.push(saved)`	서버가 돌려준 줄을 담아 둡니다
`window.confirm(...)`	떼기 전에 한 번 묻습니다
`list.splice(at, 1)`	담아 둔 목록에서 그 줄을 뺍니다
`ids.splice(from, 1)[0]`	옮길 줄의 번호를 꺼내고 목록에서 뺍니다
`ids.splice(to, 0, movedId)`	새 자리에 끼워 넣습니다
`await` 를 차례로	칸 번호가 겹치면 안 되므로 하나씩 저장합니다
`catch` 에서 다시 읽기	중간에 실패하면 서버 값으로 화면을 되돌립니다

4. 왜 이렇게

선택	이유
붙일 자리를 이어 붙임	새 카드가 늘 끝에 오는 것이 자연스럽습니다 (FR-263)
빈 칸 100 을 거쳐 옮김	`UQ(track_id, slot_index)` 때문에 자리를 맞바꿀 수 없습니다
하나씩 차례로 저장	두 줄이 같은 칸 번호를 잠깐이라도 함께 가질 수 없습니다
중복 규칙 방어	붙일 때도 `top + 1` 이 쓰이면 빈 칸을 찾습니다
떼기 전에 묻기	되돌릴 수 없는 동작입니다

5. 시험

시험 프로젝트에서 이펙트를 붙였다가(`Compressor → EQ3 → Gain`) 그 카드를 떼어 원래 2장으로 되돌렸습니다. 순서 바꾸기 시험에서 EQ3 를 앞으로 끌어 `slot_index` 가 1 에서 0 으로 바뀌고 Compressor 가 1 이 된 것을 `GET /tracks/4639/plugins` 로 확인했습니다.

JavaScript 261 — 상세 설정 모달 (`openDeviceModal` · `renderModalTabs` · `paramGroups` · `renderModalParams` · `paramCellHtml` · `bindModalParam` · `setDraft` · `markDirty` · `saveDeviceModal` · `updateSideButtons` · `closeDeviceModal`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 카드를 눌러 연 상세 설정 창을 다룹니다 (FR-2181 · FR-2187 · FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	260 붙이기 · 고치기 · 떼기 · 순서 · 262 프리셋과 켜고 끄기

1. 하는 일

카드 몸통을 누르면 상세 설정 창이 열립니다. 창에는 그룹 탭, 그 그룹의 모든 파라미터(노브 · 셀렉트 · 토글), 프리셋 고르기, A/B 비교, 초기화 버튼이 있습니다. 값은 여기서는 **초안**(`modal.draft`)에만 담고, Save 를 눌러야 서버에 저장합니다. 저장할 때 그 전 값을 B 쪽에 담아 두어 A/B 비교로 되돌릴 수 있게 합니다. `markDirty` 는 `Unsaved changes` 표시를 맡습니다.

```
openDeviceModal        base(지금 값) · draft(고칠 값) · alt(이전 값)
   ├─ renderModalTabs(paramGroups(plugin))
   ├─ renderModalParams()      paramCellHtml + bindModalParam
   ├─ loadPresets → fillPresetSelect
   └─ Save → saveDeviceModal → updateDevice(PATCH) → refreshCells
```

2. 언제 쓰나

쓰는 곳	
`wireCells` (263)	카드 클릭에서 `openDeviceModal` 을 부릅니다
`wireDialogs` (263)	탭 · A/B · Save · 초기화 버튼을 배선합니다
`removeDevice` (260)	뗀 장치의 창이 열려 있으면 닫습니다

3. 코드

```
export function openDeviceModal(track, kind, device) {
  const plugin = pluginById(device.plugin_id);
  if (!plugin) return;
  const base = withDefaults(plugin, device.params);
  modal = {
    track: track,
    kind: kind,
    device: device,
    plugin: plugin,
    base: base,
```

```
    alt: device.__alt || null,
    draft: withDefaults(plugin, device.params),
    sideB: false,
    dirty: false,
  };
  const groups = paramGroups(plugin);
  modal.group = groups[0];
  setText('device-art', artHtml(plugin).replace(/<[^>]*>/g, ''));
  setText('device-title', plugin.name);
  setText('device-sub', (kind === 'INSTRUMENT' ? 'Instrument' : 'Effect') + ' · slot ' + device.slot_ind
ex + ' · ' + plugin.engine);
  renderModalTabs(groups);
  renderModalParams();
  loadPresets(plugin).then(fillPresetSelect).catch(warn);
  markDirty(false);
  const dlg = document.getElementById('device-modal');
  if (dlg && dlg.showModal) dlg.showModal();
  updateSideButtons();
}

/** param_schema 의 group 을 처음 나온 차례대로 (없으면 General) */
function paramGroups(plugin) {
  const out = [];
  ((plugin && plugin.param_schema) || []).forEach(function (item) {
    const g = item.group || 'General';
    if (out.indexOf(g) < 0) out.push(g);
  });
  return out.length ? out : ['General'];
}

/** 모달의 파라미터 칸 그리기 (FR-2187) — 노브 · 셀렉트 · 토글을 param_schema 대로 자동 생성. */
export function renderModalParams() {
  const box = document.getElementById('device-params');
  if (!box || !modal) return;
  const schema = (modal.plugin.param_schema || []).filter(function (item) {
    return (item.group || 'General') === modal.group;
  });
  box.innerHTML = schema.map(function (item) { return paramCellHtml(item); }).join('')
    || '<p class="sq-cell__hint">This plugin has no parameters.</p>';
  schema.forEach(function (item) {
    const cell = box.querySelector('[data-param-cell="' + item.key + '"]');
    if (cell) bindModalParam(cell, item);
  });
}

/** 파라미터 한 칸 — 노브 / 셀렉트 / 토글 */
function paramCellHtml(item) {
  const v = modal.draft[item.key];
  let control;
  if (item.type === 'select') {
    const opts = (item.options || []).map(function (o) {
      return '<option value="' + esc(o.value) + '"' + (String(o.value) === String(v) ? ' selected' : '')
+ '>' + esc(o.label) + '</option>';
    }).join('');
    control = '<select data-modal-param="' + esc(item.key) + '" aria-label="' + esc(item.name) + '">' +
opts + '</select>';
  } else if (item.type === 'toggle') {
```

```
    control = '<span class="sq-switch" role="switch" tabindex="0" data-modal-param="' + esc(item.key) +
'" aria-label="' + esc(item.name) + '" aria-pressed="' + String(!!v) + '"></span>';
  } else {
    control = '<span class="sq-knob" role="slider" tabindex="0" data-modal-param="' + esc(item.key) + '"
aria-label="' + esc(item.name) + '"'
      + ' aria-valuemin="' + numOr(item.min, 0) + '" aria-valuemax="' + numOr(item.max, 1) + '" aria-val
uenow="' + numOr(v, 0) + '">'
      + '<i style="--a:' + knobAngle(item, v) + 'deg"></i>'
      + '<span class="sq-knob__val">' + esc(fmtValue(item, v)) + '</span></span>';
  }
  return '<div class="sq-param" data-param-cell="' + esc(item.key) + '">' + control
    + '<span class="sq-param__name" title="' + esc(item.key) + '">' + esc(item.name) + '</span></div>';
}

/** 모달 저장 (FR-2187) — 초안을 params 로 PATCH. A/B 비교를 위해 저장 전 값을 B 쪽에 담아 둠 */
export async function saveDeviceModal() {
  if (!modal) return;
  const saved = await updateDevice(modal.track, modal.kind, modal.device, { params: modal.draft });
  if (!saved) return;
  modal.device.__alt = modal.base;
  modal.alt = modal.base;
  modal.base = withDefaults(modal.plugin, modal.device.params);
  modal.draft = Object.assign({}, modal.base);
  markDirty(false);
  refreshCells(modal.track);
  updateSideButtons();
  showToast(modal.plugin.name + ' saved');
}

/** A / B / 초기화 버튼 (FR-246) */
export function updateSideButtons() {
  const ab = document.getElementById('device-ab');
  if (!ab || !modal) return;
  Array.prototype.forEach.call(ab.querySelectorAll('[data-ab]'), function (b) {
    const side = b.dataset.ab;
    const on = side === 'A' ? !modal.sideB : modal.sideB;
    b.classList.toggle('is-on', on);
    b.setAttribute('aria-pressed', String(on));
  });
}
```

문법 풀이

식	뜻
`device.__alt || null`	지난 저장 때 담아 둔 이전 값입니다
`groups[0]`	처음 탭을 열어 둡니다
`out.indexOf(g) < 0`	처음 나온 그룹만 담아 차례를 지킵니다
`(item.group || 'General')`	그룹이 없는 파라미터는 General 로 묶습니다
`' selected'`	지금 값과 같은 옵션을 골라 둡니다
`item.type === 'select' | 'toggle' | 그 밖`	세 가지 모양을 만듭니다
`modal.base = withDefaults(...)`	저장 뒤 기준값을 새 값으로 옮깁니다
`side === 'A' ? !modal.sideB : modal.sideB`	A 와 B 중 켜진 쪽을 표시합니다

4. 왜 이렇게

선택	이유
초안에만 담기	끄는 동안 서버에 요청이 쏟아지지 않고, 되돌리기도 쉽습니다
그룹 탭	Compressor 는 Main 과 Sidechain 처럼 파라미터가 많습니다
저장 전 값을 B 에 담기	A/B 비교가 저장 한 번으로 자연스럽게 동작합니다 (FR-246)
스키마로 자동 생성	플러그인이 늘어도 화면 코드를 고치지 않아도 됩니다 (FR-2187)
노브는 기존 조작 재사용	카드와 모달의 손맛이 같아집니다

5. 시험

Compressor 카드를 눌러 창 제목 `Compressor`, 탭 `Main`, 파라미터 6칸이 나오는 것을 확인했습니다. 노브를 돌려 `Unsaved changes` 가 뜨고 Save 를 누르면 `Saved` 로 바뀌는 것, 저장 뒤 B 를 누르면 이전 값으로, A 를 누르면 새 값으로 돌아오는 것을 확인했습니다.

JavaScript 262 — 프리셋과 켜고 끄기 (`loadPresets` · `fillPresetSelect` · `applyPreset` · `toggleBypass` · `toggleSolo`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 프리셋을 고르고, 바이패스와 겹 솔로를 켭니다 (FR-246 · FR-263 · FR-245)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	261 상세 설정 모달 · 263 이벤트 배선과 잔손질

1. 하는 일

`loadPresets` 는 플러그인의 프리셋 목록을 `GET /plugins/{id}/presets` 로 한 번만 읽어 기억합니다. `fillPresetSelect` 는 그 목록으로 고르기 상자를 채웁니다. `applyPreset` 은 고른 프리셋 값을 초안에 올리고 저장되지 않은 상태로 표시합니다. 프리셋 자체가 서버에 저장되는 것이 아니라, 값을 초안에 올려 두었다가 Save 로 저장하는 방식입니다. `toggleBypass` 와 `toggleSolo` 는 카드의 버튼을 눌렀을 때 그 장치 값 하나를 뒤집어 저장합니다.

2. 언제 쓰나

쓰는 곳	
`openDeviceModal` (261)	창을 열 때 `loadPresets` 를 부릅니다
`wireDialogs` (263)	프리셋 적용 버튼과 카드 버튼을 배선합니다
`wireCells` (263)	바이패스 · 솔로 버튼 클릭에서 두 토글을 부릅니다

3. 코드

```
export async function loadPresets(plugin) {
  if (presets[plugin.plugin_id]) return presets[plugin.plugin_id];
  const list = await api.get('/plugins/' + plugin.plugin_id + '/presets');
  presets[plugin.plugin_id] = list || [];
  return presets[plugin.plugin_id];
}

/** 프리셋 고르기 <select> 채우기 */
function fillPresetSelect(list) {
  const sel = document.getElementById('device-preset');
  if (!sel) return;
  sel.innerHTML = '<option value="">— Init —</option>' + (list || []).map(function (p) {
    return '<option value="' + p.preset_id + '">' + esc(p.name) + ' (' + esc(p.source) + ')</option>';
  }).join('');
}

export function applyPreset(presetId) {
  if (!modal) return;
```

```
  const list = presets[modal.plugin.plugin_id] || [];
  const found = list.filter(function (p) { return String(p.preset_id) === String(presetId); })[0];
  if (!found) {
    modal.draft = defaultParams(modal.plugin);
    modal.device.__preset = null;
  } else {
    modal.draft = withDefaults(modal.plugin, found.params);
    modal.device.__preset = found.name;
  }
  markDirty(true);
  renderModalParams();
}

/** 이펙트 카드의 바이패스 켜기/끄기 (FR-263) */
export function toggleBypass(track, card) {
  const device = deviceOfCard(track, card);
  if (!device) return;
  const next = !device.is_bypassed;
  updateDevice(track, 'EFFECT', device, { is_bypassed: next }).then(function (saved) {
    if (saved) refreshCells(track);
  });
}

/** 악기 겹 솔로 켜기/끄기 (FR-245) */
export function toggleSolo(track, card) {
  const device = deviceOfCard(track, card);
  if (!device) return;
  const next = !device.is_solo;
  updateDevice(track, 'INSTRUMENT', device, { is_solo: next }).then(function (saved) {
    if (saved) refreshCells(track);
  });
}
```

문법 풀이

식	뜻
`if (presets[plugin.plugin_id]) return ...`	이미 읽은 목록은 다시 묻지 않습니다
`p.source`	프리셋이 어디서 왔는지(공장 · 사용자)를 보여 줍니다
`String(p.preset_id) === String(presetId)`	고른 값이 글자로 와도 맞춰 봅니다
`modal.draft = withDefaults(...)`	프리셋 값을 기본값 위에 깔아 초안으로 만듭니다
`modal.device.__preset = found.name`	카드에 보여 줄 프리셋 이름입니다
`!device.is_bypassed`	지금 상태를 뒤집습니다
`.then(function (saved) { if (saved) ... })`	저장이 성공했을 때만 다시 그립니다

4. 왜 이렇게

선택	이유
프리셋을 초안에만 올림	값을 확인하고 Save 로 확정하는 흐름이 자연스럽습니다
Init 을 첫 항목으로	프리셋을 고르지 않은 상태로 되돌릴 수 있습니다
목록을 기억함	창을 열 때마다 서버에 묻지 않습니다

선택	이유
바이패스 · 솔로를 따로	켜고 끄는 값이 서로 다르고 즉시 저장해야 소리 결과가 바로 바뀝니다

5. 시험

시험 프로젝트에서 Compressor 카드의 바이패스 버튼을 눌러 카드에 `is-bypassed` 가 붙고 `GET /tracks/4639/plugins` 의 `is_bypassed` 가 `true` 로 저장되는 것을 확인했습니다. 프리셋 목록은 시드 프리셋 0건이라 `— Init —` 한 줄만 나왔고, Init 을 고르면 값이 기본값으로 돌아오는 것을 확인했습니다.

JavaScript 263 — 이벤트 배선과 잔손질 (`wireCells` · `wireChainDrag` · `wireDialogs` · `wireDemoControls` · `startMeters` · `setText` · `warn` · `errText` · `esc`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 칸 · 창 · 예시 화면의 입력을 받아 함수로 잇습니다 (FR-244 · FR-263)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	262 프리셋과 켜고 끄기 · S1-10 갈래 · 커넥트 링크(예정)

1. 하는 일

`wireCells` 는 트랙 목록에 클릭 하나를 걸어 두고, 눌린 곳에 따라 알맞은 함수를 부릅니다. 트랙 머리 버튼이면 저장, V 면 칸 채우기, 카드 안 버튼이면 붙이기 · 바이패스 · 떼기 · 겹 솔로, 카드 몸통이면 상세 모달입니다. 이때 펼침 칸이 트랙 줄의 **형제**라 줄이 잡히지 않는 문제를 한 줄로 해결합니다(칸 앞의 줄을 트랙 줄로 씀).
`wireChainDrag` 는 이펙트 카드를 끌어 순서를 바꾸고, `wireDialogs` 는 창 버튼을, `wireDemoControls` 는 예시 화면 조작을 맡습니다.

2. 언제 쓰나

쓰는 곳	
`initDevices` (243)	페이지가 뜰 때 세 배선 함수를 모두 부릅니다
`renderCells` (251) · `saveDeviceModal` (261)	값이 바뀌면 화면을 다시 그려 배선이 새 요소에 붙습니다

3. 코드

```
function wireCells(lanes, onChange) {
  if (!lanes || lanes.dataset.devWired === '1') return;
  lanes.dataset.devWired = '1';
  lanes.addEventListener('click', function (e) {
    const b = e.target.closest('button');
    const acc = e.target.closest('.sq-acc');
    const live = !!acc && acc.classList.contains('is-live');
    // 펼침 칸(.sq-acc)은 트랙 줄(.sq-track)의 형제이므로 칸 안을 누르면 줄이 잡히지 않습니다.
    // 그때는 칸 바로 앞 줄을 트랙 줄로 씁니다.
    let row = e.target.closest('.sq-track');
    if (!row && acc && acc.previousElementSibling) row = acc.previousElementSibling;
    const track = row && row.classList.contains('sq-track') ? trackById(row.dataset.trackId) : null;

    if (track && b && b.closest('.sq-track__head') && b.matches('.sq-power, .is-mute, .is-solo')) {
      window.setTimeout(function () { syncTrackHead(track); }, 0);
      onChange({ type: 'trackState', trackId: track.track_id });
      return;
    }
    if (track && b && b.matches('.sq-expand')) {
```

```
      window.setTimeout(function () { ensureLiveCells(track); }, 0);
      return;
    }
    if (!live || !track) return;

    const card = e.target.closest('.sq-fx');
    if (b) {
      if (b.classList.contains('is-dev-add')) {
        const kind = b.dataset.kind === 'INSTRUMENT' ? 'INSTRUMENT' : 'EFFECT';
        openPluginPicker(kind, function (plugin) { addDevice(track, kind, plugin); });
        return;
      }
      if (card && b.classList.contains('sq-fx__power')) { toggleBypass(track, card); return; }
      if (card && b.classList.contains('sq-fx__del')) {
        const device = deviceOfCard(track, card);
        if (device) removeDevice(track, card.dataset.deviceKind, device);
        return;
      }
      if (card && b.classList.contains('sq-irow__solo')) { toggleSolo(track, card); return; }
    }
    // 카드 몸통 · 악기 줄 이름 클릭 → 상세 설정 모달
    if (card && !e.target.closest('[role="slider"]')) {
      const device = deviceOfCard(track, card);
      if (device) openDeviceModal(track, card.dataset.deviceKind, device);
      return;
    }
    onChange({ type: 'deviceClick' });
  });
  wireChainDrag(lanes);
}

/** 재생 중일 때만 미터를 흔드는 타이머 시작 */
export function startMeters() {
  if (meterTimer) return;
  if (typeof window === 'undefined' || !window.setInterval) return;
  meterTimer = window.setInterval(tickMeters, 140);
}

function tickMeters() {
  const bars = document.querySelectorAll('.sq-acc.is-live .sq-meter i, .sq-strip.is-live .sq-meter i');
  if (!bars.length) {
    window.clearInterval(meterTimer);
    meterTimer = 0;
    return;
  }
  const playing = isPlaying();
  Array.prototype.forEach.call(bars, function (bar) {
    const now = parseFloat(bar.style.getPropertyValue('--m')) || 0;
    const next = playing
      ? Math.max(4, Math.min(96, now + (Math.random() * 34 - 14)))
      : Math.max(0, now - 9);
    bar.style.setProperty('--m', Math.round(next) + '%');
  });
}

/** HTML 에 넣을 글 안전하게 */
function esc(t) {
```

```
  return String(t == null ? '' : t).replace(/[&<>"]/g, function (ch) {
    return { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;' }[ch];
  });
}

/** 오류 알림 + 콘솔 */
function warn(err) {
  showToast(errText(err));
  if (window.console) console.warn('[devices]', err);
}

/** 오류 → 보여 줄 글 */
function errText(err) {
  if (err instanceof ApiError) return err.message || ('Request failed (' + err.status + ')');
  return 'Something went wrong — please try again';
}
```

문법 풀이

식	뜻
`lanes.dataset.devWired === '1'`	배선을 두 번 걸지 않습니다
`e.target.closest('.sq-acc')`	누른 곳에서 가장 가까운 펼침 칸을 찾습니다
`row = acc.previousElementSibling`	칸 바로 앞 줄을 트랙 줄로 씁니다
`b.matches('.sq-power, .is-mute, .is-solo')`	여러 이름 중 하나라도 맞는지 봅니다
`window.setTimeout(..., 0)`	arrange 가 화면을 먼저 바꾼 뒤 값을 읽게 합니다
`!e.target.closest('[role="slider"]')`	노브를 누른 것은 모달 열기로 보지 않습니다
`window.setInterval(tickMeters, 140)`	0.14초마다 미터를 흔듭니다
`bar.style.getPropertyValue('--m')`	지금 막대 길이를 읽습니다
`{ '&': '&', ... }[ch]`	글자를 안전한 표기로 바꿉니다

4. 왜 이렇게

선택	이유
클릭 한 곳에서 갈라 처리	요소마다 배선하면 다시 그릴 때 빠뜨리기 쉽습니다
형제 줄 문제를 여기서 해결	펼침 칸은 트랙 줄 안이 아니라 옆에 있습니다
다음 틱에 읽기	arrange 의 화면 변경이 먼저 끝나야 값이 맞습니다
미터를 흉내 냄	소리 값은 다음 작업(FR-260 대)이라 지금은 움직임만 보여 줍니다
`esc` 로 모두 감쌈	사용자가 정한 이름이 화면을 깨뜨리지 않게 합니다

5. 시험

시험 프로젝트에서 트랙 펼치기, 카드 몸통 클릭으로 모달 열기, 바이패스 · 떼기, 이펙트 끌어 순서 바꾸기, 고르기 창에서 플러그인 붙이기가 모두 동작하고 자바스크립트 오류가 0건인 것을 확인했습니다. 재생 중에만 미터가 흔들리고 멈추면 잦아드는 것도 확인했습니다.